

Article

Resource Scheduling Algorithm for Edge Computing Networks Based on Multi-Objective Optimization

Wenrui Liu ^{1,2}, Jiale Zhu ³ , Xiangming Li ^{2,4,*}, Yichao Fei ¹, Hai Wang ¹, Shangdong Liu ⁵ , Xiaoyao Zheng ³ and Yimu Ji ⁵

¹ China Tower Corporation Limited, Beijing 100195, China; liuw@chinatowercom.cn (W.L.); feiyc3@chinatowercom.cn (Y.F.)

² School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China

³ School of Computer and Information, Anhui Normal University, Wuhu 241003, China; jlz@ahnu.edu.cn (J.Z.); zxiaoyao@ahnu.edu.cn (X.Z.)

⁴ Aerospace and Informatics Domain, Beijing Institute of Technology, Zhuhai 519008, China

⁵ School of Computer Science, Nanjing University of Posts & Telecommunications, Nanjing 210023, China; lsd@njupt.edu.cn (S.L.); jiy@njupt.edu.cn (Y.J.)

* Correspondence: xmli@bit.edu.cn

Abstract

Edge computing networks represent an emerging technological paradigm that enhances real-time responsiveness for mobile devices by reallocating computational resources from central servers to the network's edge. This shift enables more efficient computing services for mobile devices. However, deploying computing services on inappropriate edge nodes can result in imbalanced resource utilization within edge computing networks, ultimately compromising service efficiency. Consequently, effectively leveraging the resources of edge computing devices while minimizing the energy consumption of terminal devices has become a critical issue in resource scheduling for edge computing. To tackle these challenges, this paper proposes a resource scheduling algorithm for edge computing networks based on multi-objective optimization. This approach utilizes the entropy weight method to assess both dynamic and static metrics of edge computing nodes, integrating them into a unified computing power metric for each node. This integration facilitates a better alignment between computing power and service demands. By modeling the resource scheduling problem in edge computing networks as a multi-objective Markov decision process (MOMDP), this study employs multi-objective reinforcement learning (MORL) and the proximal policy optimization (PPO) algorithm to concurrently optimize task transmission latency and energy consumption in dynamic environments. Finally, simulation experiments demonstrate that the proposed algorithm outperforms state-of-the-art scheduling algorithms in terms of latency, energy consumption, and overall reward. Additionally, it achieves an optimal hypervolume and Pareto front, effectively balancing the trade-off between task transmission latency and energy consumption in multi-objective optimization scenarios.

Keywords: computing power scheduling; edge computing networks; multi-objective optimization; resource allocation



Academic Editor: George Drosatos

Received: 9 September 2025

Revised: 1 October 2025

Accepted: 6 October 2025

Published: 9 October 2025

Citation: Liu, W.; Zhu, J.; Li, X.; Fei, Y.; Wang, H.; Liu, S.; Zheng, X.; Ji, Y. Resource Scheduling Algorithm for Edge Computing Networks Based on Multi-Objective Optimization. *Appl. Sci.* **2025**, *15*, 10837. <https://doi.org/10.3390/app151910837>

Copyright: © 2025 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and

conditions of the Creative Commons

Attribution (CC BY) license

(<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

With the rapid advancement of 5G and mobile internet technologies, intelligent terminal devices have undergone significant evolution, leading to an exponential growth in

network data traffic [1–3]. This surge has placed increasing pressure on network infrastructure, particularly as emerging applications—such as smart grid systems, autonomous driving, vehicle-to-everything communication, interactive gaming, and ultra-high-definition video streaming—demand secure, reliable, and ultra-low-latency computing services [4]. Traditional cloud computing architectures [5] struggle to meet these stringent requirements, especially in latency-critical scenarios, due to the inherent delays associated with centralized data processing. To address these challenges, there is a growing imperative to move computation closer to end users by shifting from centralized cloud data centers to the network edge. This architectural transformation aims to minimize transmission latency, reduce bandwidth consumption, and improve service responsiveness. In this context, edge computing [6,7] has emerged as a pivotal paradigm. As a distributed computing model, edge computing brings computational resources and data processing capabilities to edge nodes in proximity to data sources. By enabling real-time computation and local data analysis, it effectively reduces reliance on remote cloud servers, thereby achieving lower latency, enhanced bandwidth efficiency, and improved quality of service for delay-sensitive applications.

With the widespread deployment of edge computing and intelligent terminal devices, users can more easily access vastly distributed computing resources. However, this convenience increases the burden on edge networks and intensifies demand for computing power. In edge computing, task execution latency primarily depends on two factors: data transmission time and computation time at edge nodes [8,9]. Energy consumption comprises three components: server static energy, computational energy, and communication energy. Static energy is tied to server runtime; computational energy depends on data processing volume; and communication energy is influenced by transmission distance and data size. To minimize both latency and energy consumption, effective resource allocation between mobile devices and edge networks is critical [10]. Resource scheduling in edge nodes is typically formulated as a mixed-integer nonlinear programming problem. Traditional methods such as dynamic programming [11] and game theory [12] are computationally intensive and ill-suited for real-time decisions. To reduce complexity, heuristic local search [13] and convex optimization [14] have been applied, but they often require extensive iterations, limiting scalability. Deep reinforcement learning has shown great promise in addressing these challenges. Integration of deep Q-networks, asynchronous advantage actor–critic, and temporal difference learning [15–17] into edge scheduling has improved service quality and resource utilization. Although some studies adopt a multi-objective perspective, many scheduling strategies focus on single objectives, lack granularity, and rely predominantly on static metrics to assess computing power—failing to accurately reflect the dynamic capabilities of edge nodes. Meanwhile, with the rise of cloud-native and edge computing, microservice architecture has become the dominant paradigm for modern distributed applications. It decouples complex logic, enhancing scalability, maintainability, and enabling fine-grained resource scheduling. In real-world computing power networks and edge intelligence scenarios, applications are often structured as workflows, forming directed acyclic graphs (DAGs) with data dependencies. For instance, the ETSI MEC standard models edge applications as DAGs of service components to support dynamic orchestration across cloud, edge, and end devices [18]. Huawei’s Computing Power Network White Paper proposes abstracting tasks into DAGs composed of processing units—such as preprocessing, inference, and postprocessing—for efficient coordination of heterogeneous resources [19]. Platforms like AWS Panorama allow developers to define computer vision pipelines as DAGs, where each node represents an independently deployable compute unit [20]. Inspired by these practices, this paper models computing tasks as microservice-based DAGs to better capture the structural and resource characteristics of real-world edge applications.

Moreover, modern computing services are inherently modular, allowing decomposition into interdependent microservices [21] for more flexible and efficient scheduling. Thus, there is a pressing need for new scheduling strategies that align with the evolving nature of computing services.

To address these challenges, this paper proposes MOOECN, a multi-objective optimization algorithm for resource scheduling in edge computing networks. MOOECN accurately evaluates node computing power and enables fine-grained, dynamic resource allocation. A microservice-based edge computing model is introduced, decomposing services into independently deployable units. A hybrid metric combining static and dynamic indicators is designed to assess node capability, enabling precise microservice-to-node matching. The scheduling problem is formulated as a multi-objective Markov decision process and solved using multi-objective reinforcement learning with proximal policy optimization, simultaneously minimizing service delay and energy consumption in dynamic environments.

Our contributions are as follows:

- To assess the computing power of heterogeneous edge nodes, this paper proposes a hybrid computing power measurement method that combines static and dynamic metrics to establish a unified evaluation system, enhancing the matching efficiency between computing nodes and service requirements.
- To facilitate real-time scheduling of microservices for computing power services in edge computing network scenarios, this paper presents a multi-objective optimization model for microservice scheduling in edge computing networks, targeting minimized latency and energy consumption. Formulated as a MOMDP problem, it is efficiently solved via MORL and PPO algorithms, enabling dynamic multi-objective resource allocation.
- We conducted extensive simulation experiments to validate the effectiveness and feasibility of the proposed multi-objective optimization-based resource scheduling algorithm for edge computing networks. The results demonstrate that our algorithm outperforms others in terms of comprehensive rewards for latency and energy consumption, as well as achieving an optimal Pareto front and hypervolume.

2. Related Work

2.1. Edge Computing Power Scheduling Strategy

Edge computing scheduling primarily involves task offloading and allocation. Task offloading transfers computation from the cloud or local devices to edge nodes, alleviating resource pressure on central and terminal systems. Task allocation assigns workloads to appropriate edge nodes to optimize performance, latency, energy consumption, and other key metrics.

A fundamental challenge in computation offloading lies in determining the most effective strategy. Lan et al. [22] proposed a task partitioning and coordination framework tailored to heterogeneous edge platforms, specifically designed for computer vision applications, which considers the partitioning and coordinated utilization of CPU and GPU resources in heterogeneous edge environments. To address excessive delays during offloading that can significantly degrade user experience, Chen et al. [23] optimized the average response time of multi-task parallel scheduling. Pu et al. [24] utilized the Lyapunov method to develop an online task scheduling algorithm aimed at minimizing the system's average energy consumption. Jiao et al. [25] focused on task scheduling strategies that minimize both latency and energy costs in a multi-user, single-edge-node scenario. Wang et al. [26] introduced a decentralized computation offloading algorithm for training tasks, which reduces the distribution gap in observation–action pairs across multiple expert agents, thereby effectively mimicking expert behavior. Zhang et al. [27] proposed an energy-saving offloading strategy based on a genetic operator adaptive particle swarm

optimization algorithm, constructing a comprehensive system energy consumption model that incorporates cloud-edge server operation time, switching energy, and computational energy, thus facilitating the deployment of deep neural networks in energy-constrained edge networks. Gao et al. [28] proposed an innovative offloading strategy that considers task dependency, task priority, and resource consumption from the perspective of multi-dimensional dependencies between server clusters and tasks, employing multi-agent reinforcement learning to enhance offloading performance and advance the state of the art in edge computing.

Due to the heterogeneity and resource constraints of edge resources, scheduling in edge networks poses greater challenges than centralized scheduling in cloud data centers. To achieve high real-time performance, Peng et al. [29] proposed a decentralized approach for online task scheduling and resource allocation in edge IoT environments, significantly reducing offloading response times and improving resource utilization. Phan et al. [30] introduced a traffic-aware horizontal pod autoscaler based on Kubernetes, which performs resource scaling for IoT applications using real-time network traffic information from nodes, effectively reducing application response time and increasing throughput. In edge computing power networks, considering the random arrival of tasks, service migration delays, and queuing times for unprocessed tasks, Liu et al. [31] proposed a parameterized deep Q-network method to jointly optimize service placement and computational resource allocation, aiming to minimize total task delay. Zhou et al. [32] presented an approach combining mixed-integer nonlinear programming with double deep Q-networks to jointly optimize computation offloading and resource allocation in dynamic multi-user MEC systems, targeting minimal overall system energy consumption. Zhang et al. [33] proposed an online parallel task scheduling algorithm that builds adaptive threshold structures on each edge server, enabling scheduling decisions based on system states and active user information. Khoshvaght et al. [34] introduced a self-supervised deep reinforcement learning framework for MEC environments, which generates task embeddings via self-supervised learning and achieves efficient task allocation under dynamic conditions by integrating contrastive learning with policy optimization. Long et al. [35] proposed a security-aware DAG task scheduling strategy with a security trust model to minimize task completion time while satisfying security constraints. Xie et al. [36] presented a task scheduling approach that integrates heterogeneous resources and supports dynamic collaboration, designing a sliding window-based dynamic scheme for real-time task prioritization and optimal matching to suitable resource nodes.

The aforementioned methods primarily rely on a single static metric to assess node computing power, failing to capture their comprehensive capabilities. Moreover, many approaches decompose or transform multi-objective optimization problems into single-objective ones, resulting in scheduling strategies with narrow and coarse-grained objectives. This simplification limits their ability to accurately reflect the true computing power landscape in edge computing networks. Table 1 summarizes the differences among various edge computing resource scheduling strategies.

Table 1. Comparison of different edge computing resource scheduling algorithms.

Algorithm Name	Algorithm Type	Task Type	Multi-Objective	Dynamic Scheduling	Lightweight
EDGEVISION [21]	Heuristic Rules + Heterogeneous Resource Abstraction Model	DAG Tasks	✓	✗	✓
COTDCEG [23]	Greedy + Genetic Algorithm	DNN Tasks	✓	✗	✓
D2D Fogging [24]	Online Algorithm Based on Stochastic Optimization	Mobility-Aware Tasks	✓	✓	✓
EMCOM [25]	Game Theory + Convex Optimization	Mobility-Aware Tasks	✓	✗	✓

Table 1. Cont.

Algorithm Name	Algorithm Type	Task Type	Multi-Objective	Dynamic Scheduling	Lightweight
SPSO-GA [27]	RL + Q-learning	DNN Tasks	✓	✓	✓
Com-DDPG [28]	MADDPG	IoV Tasks	✓	✓	✗
DoSRA [29]	Distributed Online RL	Heterogeneous Tasks	✓	✓	✓
THPA [30]	Traffic-Aware Algorithm	Microservice Tasks	✓	✓	✓
PDQN [31]	DQN + Policy Gradient	Microservice Tasks	✓	✓	✓
DRLMC [32]	DQN + Policy Gradient	Mobility-Aware Tasks	✓	✓	✗
EdgeOPT [33]	Online Competitive Algorithm	Monolithic Tasks	✗	✓	✓
ZSTS-MEC [34]	SAC+Self-Supervised Learning	General Task Scheduling	✓	✓	✗
SecDS [35]	Heuristic Algorithm	DAG Tasks	✓	✗	✓
PATD3 [36]	TD3	General Task Scheduling	✓	✓	✗

✓ indicates that the algorithm possesses the feature, while ✗ indicates that it does not.

2.2. Multi-Objective Optimization

Optimization problems are common in engineering design, industrial planning, and production scheduling, where the goal is to minimize energy consumption and cost while maximizing profit, output, and efficiency for optimal resource allocation. Single-objective optimization (SOP) focuses on optimizing one objective under given constraints. However, real-world scenarios often involve multiple conflicting objectives and constraints—such as resources, time, and cost—necessitating multi-objective optimization (MOP) to achieve a balanced solution.

When these multi-objective problems include constraints, they are termed constrained multi-objective optimization problems (CMOPs). Traditional multi-objective evolutionary algorithms often struggle with constraint handling and require specialized techniques. A widely adopted approach converts constraints into additional objectives, enabling a unified framework to balance optimization goals and constraint satisfaction. For instance, Mokhtari et al. [37] proposed a fair scheduling method for machine learning tasks on heterogeneous edge systems, considering energy constraints while improving task on-time completion rates. Zhang et al. [38] introduced an edge-native task scheduling method to optimize the performance of edge-native applications. Ma et al. [39] analyzed three types of task constraints and developed a heuristic scheduling method based on differential evolution, achieving faster task sequencing and improved convergence. Gong et al. [40] proposed a multi-center-based prediction strategy to enhance algorithm adaptability in dynamic environments; by clustering historical optimal solutions using a penalty function, they designed a population generation mechanism that leverages spatial distribution information among predicted centers to produce an initial population with well-balanced diversity. Zheng et al. [41] proposed a collaborative fruit fly optimization method for green scheduling of unrelated parallel machines under resource constraints. Wang et al. [42] proposed a dynamic interval multi-objective evolutionary algorithm incorporating multi-task learning and inverse mapping; by combining multi-task learning with a self-evolving fuzzy system, the algorithm predicts the midpoints and widths of intervals, then uses inverse mappings to project the predicted Pareto front from objective space back into decision space, generating a well-distributed initial population for the new environment. Li et al. [43] proposed an improved multi-objective cuckoo search algorithm that integrates the single-objective cuckoo search framework with Pareto dominance, incorporating fast non-dominated sorting and crowding distance strategies to enhance solution convergence and distribution. Pan et al. [44] proposed a multi-objective clustering evolutionary algorithm (MCEA) for workflow offloading in mobile edge computing, introducing sub-deadline constraints during initialization to increase feasible solutions. An adaptive clustering mechanism is embedded in the crossover to guide individuals toward high-quality mating partners, and crossover and mutation probabilities are dynamically adjusted based on historical evolutionary information to balance convergence speed and search direction. Li et al. [45] inves-

tigated a sustainable edge computing framework modeled as a multi-objective problem minimizing latency and energy consumption, proposing a two-stage hybrid multi-objective evolutionary algorithm that uses a competitive swarm optimizer in the early stages for fast convergence and a diversity-enhanced immune algorithm in later stages to preserve population diversity. Al-Bakhrani et al. [46] proposed a multi-objective adaptive learning framework for UAV-assisted edge computing, integrating multi-objective reinforcement learning (MORL), model predictive control, adaptive particle swarm optimization, and Lyapunov optimization to jointly optimize dynamic resource allocation and system stability. Qiu et al. [47] presented a coevolutionary algorithm for dynamic multi-objective optimization, designing a multi-population coevolutionary mechanism and leveraging knowledge transfer among subpopulations to improve search efficiency.

Existing multi-objective optimization algorithms struggle to be directly applied to resource scheduling in edge computing power networks due to the complexity of the environment, including heterogeneous devices, dynamic network conditions, and diverse service requirements. To address these challenges, these algorithms require enhancements and adaptations to better align with the unique characteristics of edge computing.

3. Edge Computing Scheduling Algorithms

3.1. Microservice Edge Computing Power Network Model

In edge computing power networks, dynamic deployment of microservice instances is required as computing services evolve to ensure effective utilization of network resources. As shown in Figure 1, the edge computing power network consists of multiple edge servers, each with a base station (BS) deployed nearby. The BS provides network services to the edge server and connects various terminals. A set $ES = \{es_1, es_2, \dots, es_q\}$ denotes all edge servers. For any edge server i , the resource vector is defined as $R_i = \{r_{i1}, r_{i2}, \dots, r_{ik}\}$, representing resources such as CPU, memory, and bandwidth available on the server. The main symbols and variables used in this paper are summarized in Table 2.

In edge computing power networks, this study dynamically updates task deployment in response to real-time computing requests. The computing service is decomposed into multiple atomic microservices with hierarchical dependencies, which are deployed across different edge servers. Higher-level microservices depend on the outputs of lower-level ones and cannot be executed in parallel. An efficient resource scheduling strategy is required to handle these dependencies.

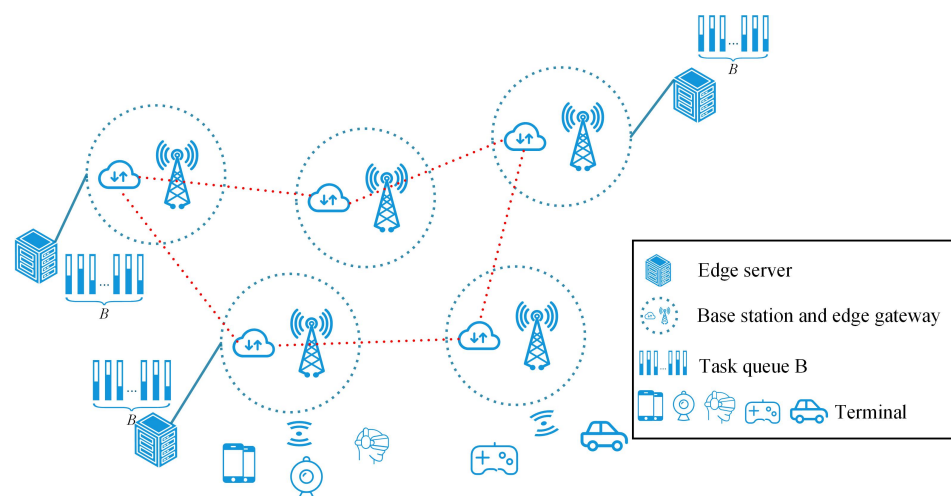


Figure 1. Microservice-based edge computing network model.

Table 2. Symbols and definitions.

Symbol	Definition
ES	Set of edge servers
R_i	Resource set of edge server i
S	Set of computing services
M_j	Set of microservices for computing service S_j
$m_{j,i}$	The i -th microservice of computing service S_j
DAG_j	Microservice dependency graph of computing service S_j
V_j	Set of all nodes in directed acyclic graph j
A_j	Set of all edges in directed acyclic graph j
$D_{j,i}$	Set of all edges incident to microservice $m_{j,i}$
M	Set of tasks pending allocation at time t
$C_{m,es}$	Transmission rate of microservice m offloaded to edge server es
T_m^{off}	Transmission delay of microservice m
E_m^{off}	Energy consumption for offloading microservice m
E_m^{exe}	Total computation energy consumption of microservice m
T_m^{exe}	Computation delay of microservice m
T_m	Total delay of microservice m
E_m	Total energy consumption of microservice m
z_{ij}	Value of the j -th computing power metric at the i -th computing node
p_{ij}	Relative importance of the j -th metric at the i -th computing node
e_j	Information entropy value of the j -th indicator
d_j	Information utility value of the j -th indicator
W_j	Weight of the j -th indicator
F_i	Comprehensive computing power evaluation value of node i
CP_{com}	Comprehensive computing capability of a computing node
CP_{total}	Total computational resources
SP_{total}	Total storage resources
CP_{idel}	Remaining resource quantity
SP_{idel}	Remaining storage space
ES_i	Computing power quintuple of node i
w	Preference vector
$s_{m,es}$	The state vector of task m offloaded to edge server es
$s_{m,u}$	The state vector of task m offloaded to terminal u
D_w	Preference w replay buffer
θ^w	Policy parameters for preference w
$x_{m,es}$	$x_{m,es}$ is a binary variable. $x_{m,es} = 1$ means task m is offloaded to edge server es and $x_{m,es} = 0$ otherwise.
L_m^s	The byte size of microservice m .
L_m^c	The floating-point operation count of microservice m .
F_e	The floating-point computing capacity of the edge server.
F_d	The floating-point computing capacity of the end device.
η_d	The energy efficiency ratio of the end device.
η_e	The energy efficiency ratio of the edge server.

The set of computing services to be deployed in the network is defined as $S = \{s_1, s_2, \dots, s_n\}$. For any computing service j , there are l microservices $M_j = \{m_{j,1}, m_{j,2}, \dots, m_{j,l}\}$. Since different edge nodes offer varying computational capabilities, it is essential to assign

appropriate edge nodes to microservices according to their resource requirements. As shown in Figure 2, we model the microservice dependency structure of service S_j as a directed acyclic graph $DAG_j = (V_j, A_j)$. Below is a detailed description of all nodes and edges in DAG_j . $V_j = \{m_{j,1}, m_{j,2}, \dots, m_{j,i}, \dots, m_{j,l}\}$ represents the set of all nodes in the graph, totaling l nodes, indicating that service S_j consists of l microservices. Each microservice has three attributes, $m_{j,i} = (c_{j,i}, CPcost_{j,i}, SPcost_{j,i})$, where $c_{j,i}$ denotes the computational load required to execute microservice $m_{j,i}$, $CPcost_{j,i}$ denotes the computing resources needed, and $SPcost_{j,i}$ denotes the storage resources required. $A_j = \{D_{j,1}, D_{j,2}, \dots, D_{j,i}, \dots, D_{j,l}\}$ is the set of edges in the graph, where $D_{j,i}$ represents all edges connected to node $m_{j,i}$. For example, if node $m_{j,5}$ has two incoming and two outgoing edges, then $D_{j,5} = \{d_{j,2,5}, d_{j,3,5}, d_{j,5,7}, d_{j,5,8}\}$. Here, $d_{j,a,b}$ denotes a directed edge from node $m_{j,a}$ to node $m_{j,b}$, whose weight indicates the amount of data that microservice $m_{j,b}$ requires from $m_{j,a}$, including $m_{j,a}$'s output and the execution environment for $m_{j,b}$. Data transmission between microservices reflects their dependency relationships.

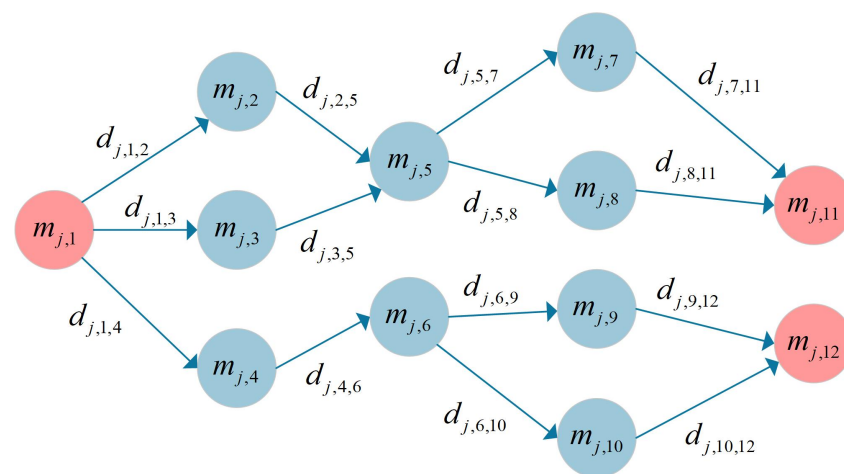


Figure 2. Microservice dependency topology graph.

At each time step t , microservices with an in-degree of 0 are selected from the DAGs of the computing service and added to the pending task set $M = \{m_1, m_2, \dots, m_l\}$, where l denotes the number of such microservices at time t . In edge computing networks, the communication link between mobile devices and edge servers is typically modeled as an additive white Gaussian noise channel. According to Shannon's information theory, the theoretical maximum transmission rate—i.e., the channel capacity—of this link is given by the Shannon–Hartley theorem [48]. The transmission rate for offloading microservice m from its local device to edge server es is given by

$$C_{m,es} = W \log_2 \left(1 + \frac{p^{\text{off}} |h_{m,es}|^2}{\sigma^2} \right), \forall m \in M, es \in ES \quad (1)$$

Here, W is the channel bandwidth, σ^2 is the noise power spectral density, representing the channel noise level, p^{off} is the offloading transmission power, and $|h_{m,es}|^2$ denotes the channel gain, reflecting the signal transmission quality from the user to the edge node.

We use $x = \{x_{m,es}\}_{m \in M, es \in ES}$ to represent the offloading decision. $x_{m,es}$ is a binary variable (taking only the values 0 or 1). When $x_{m,es} = 1$, it indicates that microservice m is offloaded to edge server es . In this case, the transmission delay of microservice m is

$$T_m^{\text{off}} = x_{m,es} \frac{L_m^s}{C_{m,es}}, \quad \forall m \in M, \forall es \in ES \quad (2)$$

where L_m^s denotes the byte size of the microservice, and $C_{m,es}$ represents the transmission rate when microservice m is offloaded to edge node es . The energy consumption for offloading microservice m is given by

$$E_m^{\text{off}} = p^{\text{off}} T_m^{\text{off}}, \quad \forall m \in M \quad (3)$$

where T_m^{off} denotes the transmission latency of microservice m , and p^{off} represents the power consumption of task offloading. If microservice m is offloaded to the edge computing network, its computational delay is given by

$$T_m^{\text{exe}} = \frac{L_m^c}{F_e}, \quad \forall m \in M \quad (4)$$

where L_m^c denotes the floating-point operation count of microservice m , and F_e represents the floating-point computing capacity of the edge server. When $x_{m,es} = 0$, it indicates that microservice m is executed locally, and its computational delay is

$$T_m^{\text{exe}} = \frac{L_m^c}{F_d}, \quad \forall m \in M \quad (5)$$

where F_d denotes the floating-point computing capacity of the end device. The total energy consumption for computing microservice m is given by

$$E_m^{\text{exe}} = (1 - x_{m,es}) \cdot \frac{L_m^c}{\eta_d} + x_{m,es} \cdot \frac{L_m^c}{\eta_e}, \quad \forall m \in M, \forall es \in ES \quad (6)$$

where L_m^c denotes the floating-point computation workload of microservice m , $x_{m,es}$ indicates whether microservice m is offloaded to edge server es , η_d denotes the energy efficiency ratio of the end device, and η_e denotes the energy efficiency ratio of the edge server. Therefore, the total delay and total energy consumption of microservice m are given by

$$T_m = T_m^{\text{off}} + T_m^{\text{exe}}, \quad E_m = E_m^{\text{off}} + E_m^{\text{exe}}, \quad \forall es \in ES \quad (7)$$

3.2. Hybrid Static–Dynamic Computing Power Measurement

In edge computing power networks, generalized computing resources encompass computational capabilities (serial and parallel), network bandwidth, and storage capacity. Different services have heterogeneous requirements for these resources. This paper proposes a multi-metric computing power measurement method based on entropy weighting, which integrates both static and dynamic resource characteristics through dimensionality reduction.

To this end, we propose a multi-indicator computing power measurement method based on the entropy weight method. This method assigns weights according to the magnitude of absolute differences in an indicator's values across different devices. An indicator with large variation across devices has strong discriminative power for assessing computing capacity and thus should be assigned a higher weight. Conversely, an indicator with little variation across devices provides limited discrimination and should be assigned a lower weight.

Assume there are q edge servers and k computing power indicators, forming the original indicator data matrix:

$$\begin{pmatrix} z_{11} & \cdots & z_{1k} \\ \vdots & \ddots & \vdots \\ z_{q1} & \cdots & z_{qk} \end{pmatrix} \quad (8)$$

Here, z_{ij} represents the value of the j -th computing power indicator for the i -th computing node. In multi-indicator evaluation, different indicators vary in dimension and range, so direct use of raw data may introduce bias. Therefore, standardization is first applied to eliminate dimensional effects. This paper adopts positive indicators, and the standardization formula is as follows:

$$r_{ij} = \frac{z_{ij} - \min(z_j)}{\max(z_j) - \min(z_j)} \quad (9)$$

Here, $\min(z_j)$ and $\max(z_j)$ represent the minimum and maximum values of the j -th computing power indicator, respectively. Next, we calculate the proportion of each indicator across the computing nodes, i.e., the relative importance of the j -th indicator for the i -th computing node. The calculation formula is

$$p_{ij} = \frac{r_{ij}}{\sum_{i=1}^k r_{ij}} \quad (10)$$

($0 \leq p_{ij} \leq 1, i = 1, 2, \dots, k, j = 1, 2, \dots, n$). Subsequently, a proportion matrix for the data indicators is established, followed by the calculation of information entropy. The information entropy value for the j -th indicator is calculated as

$$e_j = -\frac{1}{\ln m} \sum_{i=1}^m p_{ij} \ln p_{ij} \quad (11)$$

Here, $\ln m$ is the normalization factor, ensuring that the entropy value falls within the range $[0, 1]$, and $p_{ij} \ln p_{ij}$ represents the information content of the i -th computing node for the j -th computing power indicator. The information utility value for the j -th indicator is given by

$$d_j = 1 - e_j \quad (12)$$

Then, based on the information utility value, the weight of each indicator is calculated. The larger the weight, the greater the importance of the indicator in the comprehensive evaluation. The weight is defined as w_i :

$$w_j = \frac{d_j}{\sum_{j=1}^k d_j} \quad (13)$$

Based on the weights of each indicator, a comprehensive evaluation of each computing node can be conducted, and the comprehensive evaluation value for each computing node is calculated as

$$E_i = \sum_{j=1}^m r_{ij} w_j \quad (14)$$

In this study, CPU frequency, parallel computing capability, serial computing capability, and storage space are used as static indicators for computing power measurement. The entropy weight method is applied to compute the weights of CPU frequency, parallel computing capability, and serial computing capability, resulting in a multi-factor static composite indicator—the comprehensive computing capability CP_{com} of the computing node. Other static indicators include the total computing resources CP_{total} and total storage resources SP_{total} . For dynamic indicators, the remaining computing resources CP_{idel} and remaining storage space SP_{idel} are considered. Based on these static and dynamic indicators, they are integrated into a unified comprehensive computing power indicator for the computing node. For example, the computing power of node i can be represented by a quintuple $ES_i = (CP_{com_i}, CP_{total_i}, SP_{total_i}, CP_{idel_i}, SP_{idel_i})$.

3.3. Multi-Objective Optimization for Resource Scheduling

To address dynamic microservice workloads in edge computing power networks and optimize resource utilization and energy efficiency, this paper proposes a computing resource scheduling algorithm based on multi-objective optimization.

In this paper, the resource scheduling problem in edge computing networks is modeled as a multi-objective Markov decision process (MOMDP). By employing multi-objective reinforcement learning (MORL) and the proximal policy optimization (PPO) algorithm, the goal of simultaneously optimizing task transmission latency and energy consumption in dynamic environments is achieved. Figure 3 illustrates the workflow of the proposed multi-objective optimization-based computing resource scheduling algorithm.

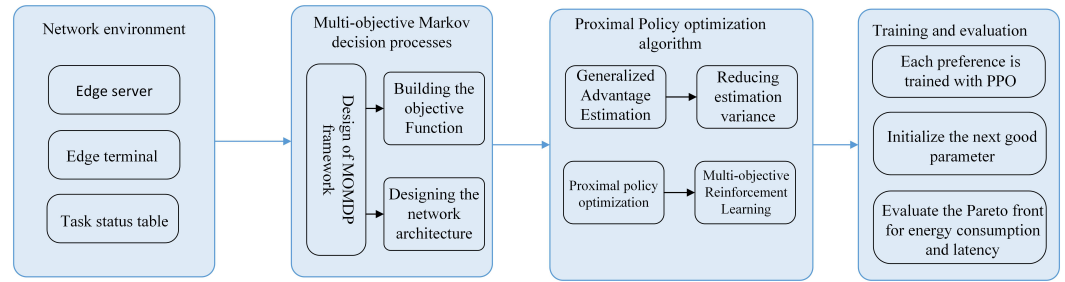


Figure 3. Multi-Objective optimization-based computing resource scheduling algorithm workflow.

Based on different application scenarios, edge computing networks have varying preferences for energy consumption and latency. Therefore, we model the problem as an MOMDP to simultaneously minimize both objectives. We employ MORL to obtain Pareto-optimal solutions under different weightings. A preference vector $w = (w_E, w_T)$ is introduced to weigh energy consumption and latency, satisfying $w_E + w_T = 1$, where E denotes energy consumption, T denotes delay, and λ^m represents the weight coefficient. For any given task m and current system state s , the policy π selects the offloading decision $x_{m,es}$ according to a probability distribution. For any given w , the multi-objective resource scheduling decision is given by the following formula:

$$\begin{aligned}
 & \min_{\pi} \quad \mathbb{E}_{\mathbf{x} \sim \pi} \left[\sum_{m \in M} \lambda^m (w_T T_m + w_E E_m) \right] \\
 & \text{s.t.} \quad x_{m,es} \in \{0, 1\}, \quad \forall m \in M, \forall es \in ES \\
 & \quad CPdemand_m \leq CPidel_{es} \\
 & \quad SPdemand_m \leq SPidel_{es}
 \end{aligned} \tag{15}$$

To facilitate multi-objective analysis, we consider a preference set $\mathcal{W} = \{w_1, w_2, \dots, w_n\}$, consisting of n distinct preferences. The system provides a corresponding policy set $\Pi = \{\pi_1, \pi_2, \dots, \pi_n\}$ to meet the multi-objective scheduling goals under different preferences. The Pareto front is used to balance the two performance metrics, and the hypervolume metric is employed to evaluate its quality.

3.3.1. MOOECN Scheduling Scheme

MOMDP is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \gamma, \mu, \mathcal{R} \rangle$, where $\mathcal{S}$ represents the state space of the system, $\mathcal{A}$ denotes the action space, $\mathcal{T}$ describes the transition process, γ is the discount factor, μ represents the probability distribution of the initial state, and $\mathcal{R}$ is the reward function. At decision step t , the system determines whether the current task m is executed locally on terminal u or offloaded to edge server es for execution.

State $\mathcal{S}$: The system considers $q + 1$ state spaces, where q denotes the number of edge servers currently online and 1 represents the terminal to which the current task belongs.

Therefore, the state $s_t \in \mathcal{S}$ at step t is a state space collection of length $q + 1$, which can be expressed as $S_t = \{s_{m,es} \cup s_{m,u} \mid es \in ES\}$. The state vector of the server at step t can be represented as

$$s_{m,es} = (L_m^s, L_m^c, C_{u,es}, F_e, n_{es}, q, \beta_{es}), \quad \forall es \in ES \quad (16)$$

Here, state $s_{m,es}$ contains the byte size L_m^s of microservice m , the floating-point computation workload L_m^c of microservice m , the transmission rate $C_{u,es}$ from the user device of task m to the edge server es , the floating-point computing capability F_e of the edge server, the number of tasks n_{es} being executed on edge server es at time t , the number of currently online edge servers q , and the residual size distribution β_e of tasks being executed on the edge server. The state vector of the terminal at step t can be expressed as

$$s_{m,u} = (L_m^s, L_m^c, F_d, n_u, u, \beta_u) \quad (17)$$

Here, the state $s_{m,u}$ contains the byte size L_m^s of microservice m , the floating-point computation workload L_m^c of microservice m , the floating-point computing capability F_d of the edge server, the number of tasks n_u being executed on end device u at time t , the user ID u to which the task belongs, and the residual size distribution β_e of tasks being executed by the end device.

Action $\mathcal{A}$: The action space is denoted as $\mathcal{A} = \{0, 1, \dots, q\}$, whose dimension is dynamically variable, depending on the number of currently online edge servers q . Here, 0 represents the user device to which the task belongs and $\{1, 2, \dots, q\}$ denotes the index of edge server es . The action $a_t \in \mathcal{A}$ at step t indicates where to schedule task m for execution. This is formally expressed as follows:

$$a_t = \sum_{es \in ES} es \cdot x_{m,es}(t) \quad (18)$$

To achieve the dynamic nature of the action space, we predefine a maximum action space $\mathcal{A}_{\max} = \{0, 1, \dots, Q_{\max}\}$, where $Q_{\max}$ denotes the maximum number of edge servers supported by the system. The output dimension of the policy network is fixed at $|\mathcal{A}_{\max}|$. At each decision step, the environment informs the agent of the current effective action dimension $|\mathcal{A}| = q + 1$. By employing a dynamic pruning mechanism, we only take the first $|\mathcal{A}|$ dimensions of the network's output logits for Softmax normalization, thereby generating a policy distribution that precisely matches the currently available resources in the environment. This design enables a single policy to adaptively respond to the dynamic changes in the resource pool, handling the joining or leaving of edge nodes without requiring retraining.

Transition $\mathcal{T}$: It describes the transition of the state space from s_t to s_{t+1} , with the action being a_t , denoted by $P(s_{t+1} \mid s_t, a_t)$.

Reward $\mathcal{R}$: Our reward function is set in vector form, given by $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}^2$. Here, r_E and r_T represent the rewards for latency and energy consumption, respectively. If the system offloads task m to server es or executes it locally on terminal u at time t , the energy consumption reward determined by the state space $s_{m,u}$ and action space a_t at this point is

$$r_E(s_t, a_t) = -\hat{E}_m = -(x_{m,es} E_m^{\text{off}} + E_m^{\text{exe}}) \quad (19)$$

where $\hat{E}_m$ is the estimated energy consumption of task m . To maximize the reward, the energy consumption is set as a negative value. The total reward for energy consumption throughout the entire training process is given by

$$R_E = \sum_{t=1}^T r_E(s_t, a_t) = - \sum_{m \in M} \hat{E}_m \quad (20)$$

The latency consumption reward at this point is

$$r_T(s_t, a_t) = -\hat{T}_m = -(x_{m,es}T_m^{\text{off}} + T_m^{\text{exe}}) \quad (21)$$

where $\hat{T}_m$ is the estimated latency of task m , also set as a negative value. The total reward for latency throughout the entire training process is given by

$$R_T = \sum_{t=1}^T r_T(s_t, a_t) = - \sum_{m \in M} \hat{T}_m \quad (22)$$

The reward after normalizing the latency and energy consumption rewards under different preferences is

$$r_T(s_t, a_t) = w^T \times \left(\frac{r_T(s_t, a_t) - \min_T}{\max_T - \min_T}, \frac{r_E(s_t, a_t) - \min_E}{\max_E - \min_E} \right) \quad (23)$$

where w^T is the transpose of w , $\min_T$, $\max_T$ are the dynamically saved minimum and maximum values of the latency reward, and $\min_E$ and $\max_E$ are the dynamically saved minimum and maximum values of the energy consumption reward. In this way, the latency and energy consumption rewards are dynamically normalized to the same scale. The total reward for the entire training process is

$$R_w = \sum_{t=1}^{T_{epi}} r_w(s_t, a_t) \quad (24)$$

3.3.2. PPO-Based Scheduling Strategy

This paper adopts an MORL strategy based on the PPO algorithm to address the resource scheduling problem in edge computing environments. By introducing a clipping mechanism to constrain the magnitude of policy updates, the training stability and sample efficiency are improved. The core idea is to use a clipping function to bound the probability ratio between the new and old policies, thereby preventing instability caused by excessively large updates. Specifically, the loss function incorporates a clipped surrogate term that penalizes updates falling outside a predefined range. The clipping function is defined as follows:

$$\begin{aligned} \nabla_{\theta} L_{clip}^i(\theta) &= \mathbb{E}_t \\ &[\min(r_{prt}(\theta), \text{clip}(r_{prt}(\theta), 1 - \epsilon, 1 + \epsilon)) \hat{A}_i(t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] \end{aligned} \quad (25)$$

where $r_{prt}(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$ is the probability ratio between the new and old policies, ϵ is the clipping hyperparameter used to limit the magnitude of policy updates, and $\hat{A}_i(t)$ is the advantage function for objective i , used to estimate the gradient direction for each objective.

This paper extends the PPO algorithm to handle multiple conflicting objectives, addressing the multi-objective optimization problem of latency and energy consumption. To reduce the variance of policy gradient estimation, the generalized advantage estimation technique is employed. By introducing a discount factor λ , the advantage function is combined to estimate the advantage value for each objective. The specific advantage function is defined as follows:

$$A_i(t) = \sum_{t'=t}^{T_{epi}-1} \gamma^{t'-t} \lambda (\alpha_i r_i(s_{t'}, a_{t'}) + \gamma V_{i,\theta}(s_{t'+1}) - V_{i,\theta}(s_{t'})) \quad (26)$$

where α_i is the weight for objective i , $r_i(s_{t'}, a_{t'})$ is the immediate reward for objective i , and $V_{i,\theta}(s_{t'+1})$ is the value function estimate for objective i .

To find the Pareto front, this paper employs the Ascent Simplex method, which generates a composite gradient direction by convexly combining multiple single-objective gradients, enabling the policy to optimize along the Pareto front. The gradient direction for preference w can be expressed as

$$\nabla_{\theta} L_{clip}^w(\theta) = w^T (\nabla_{\theta} L_{clip}^T(\theta), \nabla_{\theta} L_{clip}^E(\theta)) \quad (27)$$

where w is the weight vector of preferences, and $\nabla_{\theta} L_{clip}^T(\theta)$ and $\nabla_{\theta} L_{clip}^E(\theta)$ are the gradient directions for the latency and energy consumption objectives, respectively.

The MOOECN algorithm proposed in this paper formulates the joint minimization of latency and energy consumption as a multi-objective Markov decision process (MOMDP). The optimization objectives for latency and energy consumption are designed as reward functions for multi-objective reinforcement learning (MORL). For a given preference set $w = \{w_1, w_2, \dots, w_n\}$, the proximal policy optimization (PPO) method is employed to train the corresponding policy set $\Pi = \{\pi_1, \pi_2, \dots, \pi_n\}$, aiming to maximize the total reward R_w and approximate the Pareto front $PF(\Pi)$.

3.4. Algorithm Implementation and Analysis

3.4.1. Algorithm Implementation

We applied the proposed MOOECN method to edge computing networks, and Algorithm 1 details the workflow of the multi-objective optimization-based computing resource scheduling. First, for each preference, the replay memory buffer and policy parameters are initialized, and the learning rate and total number of training episodes are set. Subsequently, for each preference, during each training episode, actions are selected based on the current policy, and the system state is updated through the transition function. The state, action, reward, and information about the next state are stored in the memory buffer. Next, the policy parameters are updated using gradient descent, and the optimized policy is incorporated into the policy set. Finally, by computing the Pareto frontier of all policies, the optimal solution is determined.

Algorithm 1 MOOECN

- 1: Initialize the replay memory buffer D_w and policy parameters θ_w for each preference w .
 - 2: Initialize the learning rate α and the total number of episodes T_{epi} for training.
 - 3: Set the policy set $\Pi \leftarrow \emptyset$
 - 4: **for** each preference w **do**
 - 5: **for** each episode T_{epi} **do**
 - 6: **for** each step t **do**
 - 7: Select action a_t according to policy π_{θ_w}
 - 8: Obtain the next state s_{t+1} using the transition function $\mathcal{T}$
 - 9: Store $(s_t, a_t, r_w(s_t, a_t), s_{t+1})$ in D_w
 - 10: **end for**
 - 11: Update policy parameters $\theta_w : \theta_w \leftarrow \theta_w + \alpha \nabla_{\theta_w} L_w^{city}(\theta_w)$
 - 12: **end for**
 - 13: Add policy π_{θ_w} to the policy set Π
 - 14: **end for**
 - 15: Compute the Pareto front $PF(\Pi)$
-

3.4.2. Complexity Analysis

The time complexity of the algorithm is primarily determined by the training phase and the Pareto frontier computation. The time complexity of the training phase is

$O(N \cdot T_{epi} \cdot T \cdot (P + S))$, where N is the number of preferences, T_{epi} is the number of training episodes, T is the number of steps per episode, P is the number of policy parameters, and S is the state space dimension. The time complexity of the Pareto frontier computation is $O(N^2 \cdot M)$, where M is the number of objective functions. Therefore, the overall time complexity is $O(N \cdot T_{epi} \cdot T \cdot (P + S) + N^2 \cdot M)$.

4. Experimentation and Evaluation

In this chapter, we conducted extensive experiments to address the following research questions:

- Question 1: What is the performance of our multi-objective optimization-based edge computing network resource scheduling (MOOECN)?
- Question 2: How do different components of MOOECN impact its performance?
- Question 3: What is the influence of hyperparameters on MOOECN?

4.1. Experimental Setup

4.1.1. Simulation Environment

The simulation environment in this experiment mainly consists of two components: terminals and edge servers. Terminals are used to generate tasks, request task scheduling, and process tasks; edge nodes provide computational resources to handle tasks. Tasks are randomly generated by the terminals, with task sizes set within the range [0.1 MB, 100 MB], generated based on an exponential distribution and bounded by predefined upper and lower limits. This setup simulates common task load patterns in edge computing environments. The selected range reflects typical data volumes found in real-world application scenarios while preventing extreme values from adversely affecting system stability. The task size design accounts for both the frequent occurrence of small tasks and the occasional presence of large ones, aligning with the characteristics of practical task distributions in edge computing environments. At the same time, this configuration facilitates the evaluation of scheduling algorithms under various load conditions. Each task generated by the terminals is saved as a DAG composed of microservices. We set up 10 terminals and 20 edge servers and created 64 different edge computing network environments for parallel training. For each edge computing network environment, the preference set related to energy consumption and latency is set to Ω , with 50 different preferences obtained at intervals of 0.02. For each preference, a task scheduling process with 100 time steps is conducted to fit the Pareto front, and 256 edge computing network environments are created for parallel testing. The specific experimental model parameter settings are shown in Table 3.

Table 3. Model parameters.

Symbol	Quantity	Values
T_{epi}	Total number of steps in one training cycle	100
Δt	Time per step	1 s
U	Number of terminals	10
ES	Number of edge servers	20
C_{ter}	CPU frequency of terminals	2 ± 0.5 GHz
C_{edg}	CPU frequency of edge servers	4 ± 1 GHz
C	Channel bandwidth	16.6 MHz
L	Task size	0.1 MB–100 MB
p^{off}	Offloading power	0.01 W
σ^2	Noise power spectral density	−174 dBm/Hz
D	Distance between terminals and edge servers	50–500 m

In the baseline algorithms, the confidence parameter of MAB is set to 1.5; for DQN, the learning rate is 1×10^{-4} , the discount factor is 0.95, and the experience replay buffer size is 10,000; for SAC, the learning rate is 3×10^{-4} and the target network update rate is 0.005; and greedy, random, and heuristic algorithms have no trainable parameters.

4.1.2. Evaluation Metrics

We consider the following metrics to evaluate the performance of the proposed algorithm.

- Energy consumption: The total energy consumption of the computational tasks during a complete training cycle, i.e., $-\sum_{m=1}^M (x_{m,es} E_m^{\text{off}} + E_m^{\text{exe}}), es \in ES$.
- Latency: The total latency of the computational tasks during a complete training cycle, i.e., $\sum_{m=1}^M (x_{m,es} T_m^{\text{off}} + T_m^{\text{exe}}), es \in ES$.
- Total reward: The cumulative reward value obtained over a complete training cycle, i.e., $R_w = \sum_{t=1}^{T_{\text{epi}}} r_w(s_t, a_t)$.
- Pareto frontier: For any strategy under a given preference, an optimal trade-off between latency and energy consumption can be maintained, i.e., $PF(\Pi) = \{\pi \in \Pi \mid \nexists \pi' \in \Pi: y^{\pi'} \succ_P y^{\pi}\}$.
- Pareto hypervolume: This metric is used to measure the approximation quality of the Pareto frontier. It evaluates the performance of multi-objective optimization algorithms by calculating the volume between the Pareto frontier and a reference point.

4.1.3. Baseline

We evaluate the performance of the proposed MOOECN algorithm and compare it with a multi-armed bandit-based scheme, a deep Q-network-based scheme, a greedy-based scheme, a random-based scheme, a soft actor–critic-based scheme, and a heuristic-based scheme.

- Multi-armed bandit-based scheme [49]: This approach formulates the task scheduling problem in edge computing as a contextual multi-armed bandit problem. Each “arm” corresponds to an available edge server or a scheduling action (e.g., local execution, offloading to edge node A/B). At each decision step, the system observes the current task features—such as task size, deadline, and device battery level—as context information and dynamically adjusts its selection policy based on historical rewards (e.g., task completion delay, energy consumption, success rate). This approach exhibits low computational overhead and fast convergence, making it suitable for lightweight edge devices; however, it cannot explicitly model state transitions or optimize long-term cumulative rewards.
- Deep Q-network-based scheme [50]: This scheme formulates task scheduling as a Markov decision process (MDP) and employs a deep Q-network (DQN) to solve for the optimal policy. DQN uses a deep neural network to approximate the Q-function and stabilizes the training process through experience replay and a target network. This method is capable of handling high-dimensional state spaces and learning long-term optimized strategies; however, it has high sample efficiency requirements and may face significant training overhead in edge computing environments.
- Greedy algorithm-based scheme [51]: This scheme selects, at each decision step, the action that yields the highest immediate reward based solely on the current state, without considering the impact of future states. For example, it always schedules tasks to the edge node with the current lowest load or the shortest estimated completion time. It is simple to implement and highly responsive, making it suitable for scenarios with stringent real-time requirements. However, due to the lack of consideration of long-term performance, it is prone to getting trapped in local optima and tends to per-

form unstably, especially in dynamic edge environments with fluctuating workloads or resources.

- Random-based scheme [52]: This scheme selects, at each decision step, the action that yields the highest immediate reward based solely on the current state, without considering the impact of future states. For example, it always schedules tasks to the edge node with the current lowest load or the shortest estimated completion time. It is simple to implement and highly responsive, making it suitable for scenarios with stringent real-time requirements. However, due to the lack of consideration of long-term performance, it is prone to getting trapped in local optima and tends to perform unstably, especially in dynamic edge environments with fluctuating workloads or resources.
- SAC-based approach [53]: This scheme uniformly randomly selects a scheduling target from all available actions at each time step, without relying on any historical experience or state information. Although seemingly inefficient, it serves as a baseline to effectively evaluate whether other algorithms genuinely outperform random, non-strategic behavior. Moreover, in highly uncertain environments or during the early exploration phase, the random policy helps collect diverse experience data and is commonly used in the initial exploration stage of reinforcement learning algorithms.
- Heuristic algorithm-based approach [54]: This scheme designs domain-specific rules to rapidly generate approximate optimal scheduling decisions. The solving strategy, crafted based on experience, intuition, or problem-specific knowledge, aims to obtain high-quality solutions within a reasonable computational time, which is especially suitable for problems with high computational complexity that are difficult to solve exactly (e.g., NP-hard problems). While it does not guarantee finding the global optimum, it often achieves good performance in practical applications and is widely used in combinatorial optimization, scheduling, path planning, resource allocation, and related fields.

4.2. Experimental Results

4.2.1. Performance Comparison

Figure 4a illustrates the variation in total rewards under different preferences for our proposed MOOECN algorithm and the baseline algorithms. We integrated the negative values of latency and energy consumption into the reward function. A higher reward value indicates that the scheduling algorithm achieves a better balance between latency and energy consumption, aligning more closely with the optimization objectives. Our proposed MOOECN algorithm attains the highest reward values under most preference settings. Figures 4b and 4c, respectively, depict the reward variations for latency and energy consumption under different preferences for the MOOECN algorithm and other baseline algorithms. Under most preference configurations, our algorithm also achieves optimal performance in terms of both latency and energy consumption. The statistical results show that our proposed method outperforms the DQN algorithm in 41 out of 51 weight preferences and surpasses the SAC algorithm in 31 cases. The MAB algorithm performs better only under a few specific weight settings, and its performance exhibits significant fluctuations. All other baseline algorithms yield inferior results across all 51 weight preferences. Furthermore, under extreme preferences $wt = 0$ and $wt = 1$, our proposed MOOECN maintains stable performance, while several baseline methods suffer from a sharp performance degradation. This demonstrates that the proposed approach possesses superior robustness and generalization capability. MOOECN formulates the resource scheduling problem as a multi-objective Markov decision process and integrates multi-objective reinforcement

learning with the proximal policy optimization (PPO) algorithm to achieve coordinated optimization of latency and energy consumption. Its key advantage lies in effectively maintaining solution diversity and adapting to varying user preferences, thereby achieving superior overall performance in dynamic environments.

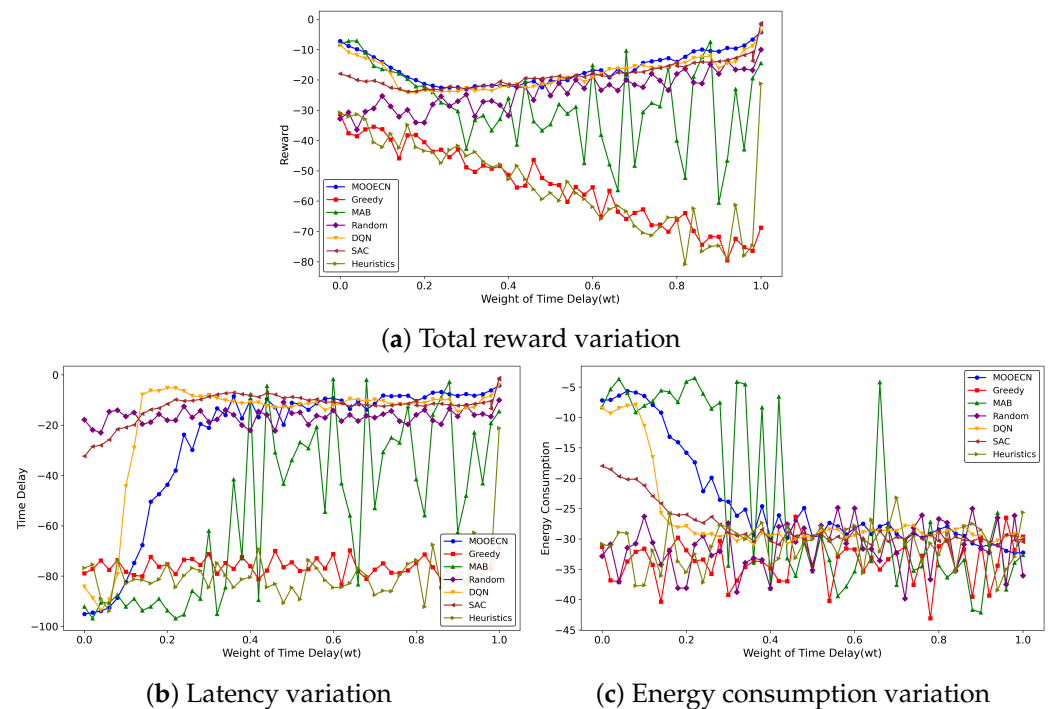


Figure 4. Comparison of reward values across different algorithms.

Simultaneously, we conducted a detailed analysis of the Pareto frontiers of each algorithm and evaluated their performance in multi-objective optimization problems using the Pareto hypervolume metric. The Pareto frontier reflects the trade-off between the two objectives of latency and energy consumption, while the Pareto hypervolume quantifies the overall performance of the algorithm. We first calculated the Pareto hypervolume for each algorithm, and the results are presented in Table 4. From the Pareto hypervolume results, it is evident that the MOOECN algorithm performs the best, with its hypervolume value being significantly higher than those of the other algorithms. This indicates that MOOECN achieves a better balance between latency and energy consumption, providing a greater number of Pareto-optimal solutions. In contrast, the random algorithm has the lowest hypervolume value, suggesting its poor performance in multi-objective optimization problems and its difficulty in finding an effective balance between latency and energy consumption. MOOECN effectively explores the optimal trade-off frontier between latency and energy consumption through multi-objective reinforcement learning, generating a more widely distributed and diverse set of non-dominated solutions. Moreover, the preference-conditioned mechanism enables the algorithm to cover the full spectrum of solutions from extreme preferences to balanced trade-offs, thereby significantly expanding the hypervolume of the Pareto front.

Table 4. Comparison of Pareto hypervolume across algorithms.

Algorithm	Greedy	Heuristics	MAB	Random	SAC	DQN	MOOECN
Pareto Hypervolume	24.22	122.63	390.39	17.34	209.62	1323.35	1545.55

Bold values indicate the best results.

Furthermore, to verify whether there exists a significant difference between our proposed method and the baseline methods, we conducted a *t*-test statistical analysis. A *p*-value ≤ 0.01 is considered statistically significant. Our method was compared with six baseline approaches, and the results are presented in Table 5. As can be observed, the proposed method exhibits statistically significant differences compared to all baseline methods.

Table 5. *p*-values of hypervolume comparison with the proposed method.

Algorithm	Greedy	Heuristics	MAB	Random	SAC	DQN
<i>p</i> -value	$\approx 6 \times 10^{-8}$	$\approx 1 \times 10^{-7}$	$\approx 4 \times 10^{-7}$	$\approx 5 \times 10^{-8}$	$\approx 2 \times 10^{-7}$	$\approx 8 \times 10^{-4}$

Through the visualization of the Pareto front, as shown in Figure 5, the performance differences among various algorithms in jointly optimizing latency and energy consumption can be intuitively evaluated. The experimental results demonstrate that the proposed MOOECN algorithm significantly outperforms existing methods in terms of solution convergence, spread, and diversity. The Pareto front obtained by MOOECN contains 21 non-dominated solutions, uniformly covering the entire trade-off region from low-latency–high-energy to high-latency–low-energy, thereby fully reflecting its capability to provide diverse scheduling strategies under different optimization preferences. Moreover, MOOECN achieves the highest hypervolume value, indicating its superior multi-objective search ability in approximating the true Pareto front and effectively balancing multiple dimensions of system performance.

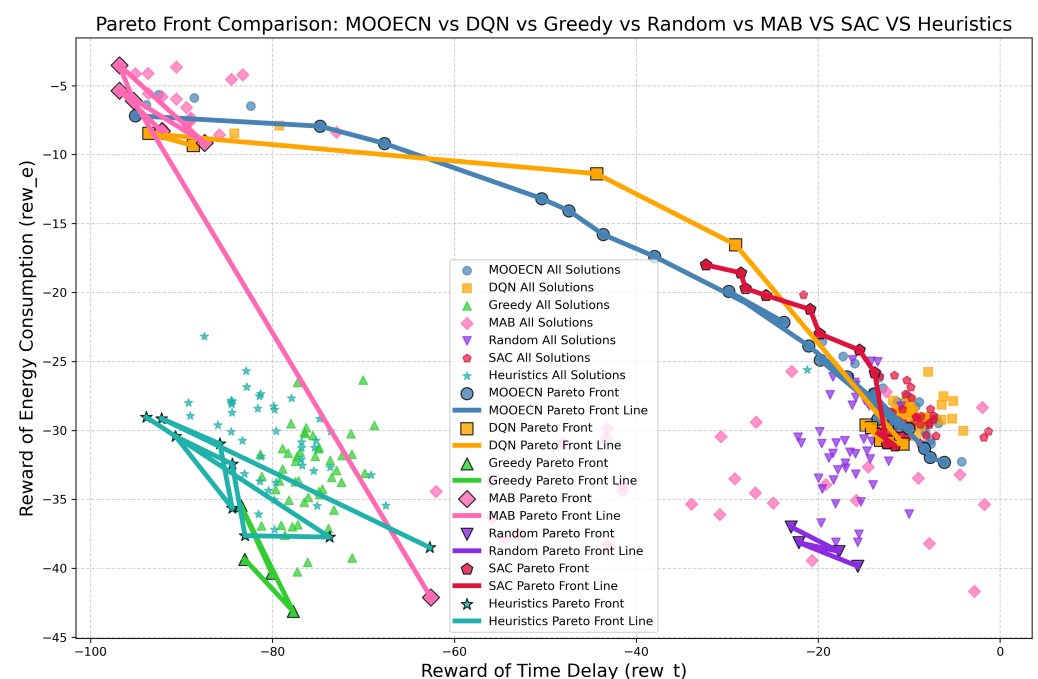


Figure 5. Privacy-preserving dynamic global graph construction.

In comparison, existing algorithms exhibit notable limitations in multi-objective optimization performance. Although DQN models complex state spaces using deep neural networks, its solution set is sparsely distributed, resulting in a relatively lower hypervolume. SAC demonstrates certain advantages in policy exploration, but experimental results show that its Pareto front is highly concentrated with a limited number of solutions, yielding a hypervolume lower than both MOOECN and DQN, indicating insufficient exploration efficiency in multi-objective tasks and an inability to adequately cover the trade-off space. MAB dynamically adjusts action selection

based on historical rewards and possesses certain online learning capabilities; however, its use of a linear model to represent state–action relationships fails to capture the high-dimensional nonlinear couplings inherent to multi-terminal, multi-server, and multi-task parallel environments. Heuristics are typically designed based on prior knowledge with fixed rules, enabling rapid generation of feasible solutions in specific scenarios, but they lack adaptive learning mechanisms and thus struggle to flexibly adjust scheduling strategies in response to the high dynamics and heterogeneity of edge environments, such as real-time load, channel conditions, and energy constraints. The greedy method prioritizes immediate reward maximization by selecting the locally optimal action at each decision step, offering low computational complexity but completely neglecting long-term system benefits, making it prone to suboptimal solutions. The random approach relies entirely on undirected action sampling without leveraging environmental feedback or any learning mechanism, leading to highly arbitrary and ineffective scheduling decisions.

In summary, existing algorithms exhibit varying degrees of limitations in terms of model expressiveness, long-term planning, multi-objective coordination, and environmental adaptability. In contrast, the proposed MOOECN algorithm consistently outperforms baseline methods—including MAB, DQN, SAC, greedy, heuristics, and random models—across key performance metrics such as Pareto front distribution, hypervolume, latency–energy trade-off capability, and cumulative reward. Experimental results demonstrate that MOOECN possesses superior multi-objective optimization capability and robustness in complex edge computing environments, enabling it to provide high-quality and diverse solutions for resource scheduling, with strong potential for practical deployment.

4.2.2. Ablation Study

To investigate the effectiveness of different components of MOOECN, we conducted an ablation study.

(1) What is the impact of the hybrid dynamic–static computing power measurement method on resource scheduling in edge computing networks? From Figure 6, it can be observed that MOOECN, which incorporates the computing power measurement scheme, achieves higher reward values in terms of energy consumption, latency, and total reward. Based on these results, we can infer that the computing power measurement method better aligns the computing resources of edge servers with the resource demands of end devices. By allocating appropriate computing nodes to computing services through the scheduling algorithm, MOOECN significantly enhances the overall efficiency of computing network scheduling.

(2) What are the differences between simple task scheduling and microservice-based computing resource scheduling? From Figure 7, it can be observed that the performance differences between simple task scheduling and microservice-based computing resource scheduling are relatively small in terms of comprehensive rewards and energy consumption. However, in terms of latency, simple task scheduling slightly outperforms microservice-based computing resource scheduling. This is primarily because microservice-based scheduling requires full consideration of the dependencies between microservices, where the execution of some microservices must wait for the completion of others, thereby increasing latency to some extent. Nevertheless, our model effectively handles more complex real-world scenarios based on microservices, demonstrating stronger adaptability and practicality.

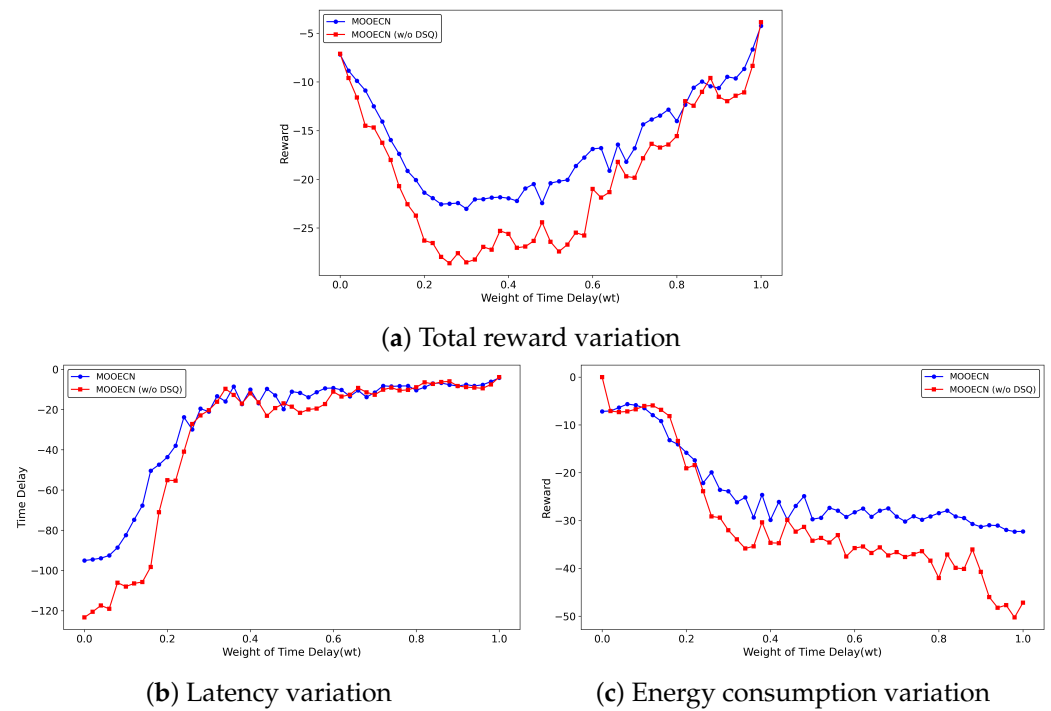


Figure 6. Effectiveness of the Hybrid Dynamic-Static Computing Power Measurement Method.

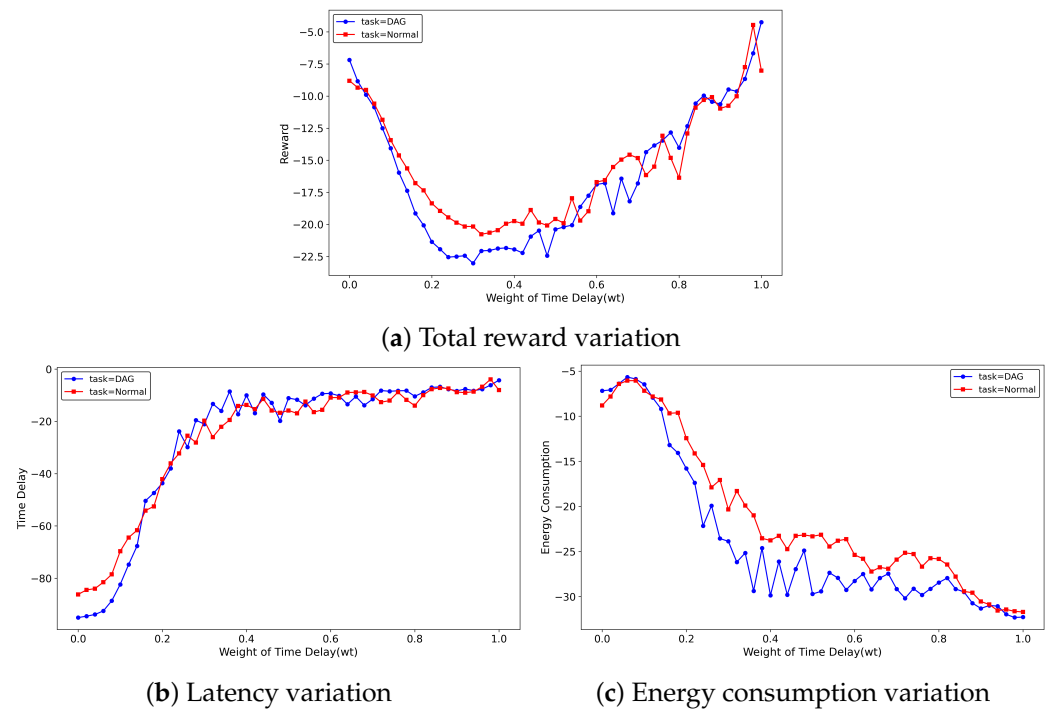


Figure 7. Effectiveness of microservice-based computing resource scheduling.

4.2.3. Hyperparameter Analysis

In this paper, we conduct hyperparameter experiments to analyze the impact of key hyperparameters on the performance of our scheduling algorithm.

(1) Impact of the number of edge servers on scheduling algorithm performance. To investigate the influence of the number of edge servers on the performance of the scheduling algorithm, we fixed the number of end devices and set the number of edge servers to edge = [10, 20, 30], testing the algorithm's performance under different configurations.

From the results in Figure 8, it can be observed that in terms of energy consumption rewards, as the number of edge servers increases, the total power consumption required to maintain all servers also rises, leading to a decrease in energy consumption rewards across all preference weights. Meanwhile, in terms of latency rewards, as the number of edge servers increases, the available computing resources become more abundant, significantly reducing task completion latency and consequently increasing the corresponding latency rewards. Regarding comprehensive rewards, when the number of edge servers is $\text{edge} = 10$, the comprehensive reward reaches its highest value in the first half of the preference weights but declines in the second half. As the number of edge servers further increases, the comprehensive reward values under different configurations gradually converge. This indicates that for different numbers of edge servers, it is necessary to select appropriate preference weights to balance task scheduling latency and energy consumption, thereby maximizing the comprehensive reward of task scheduling.

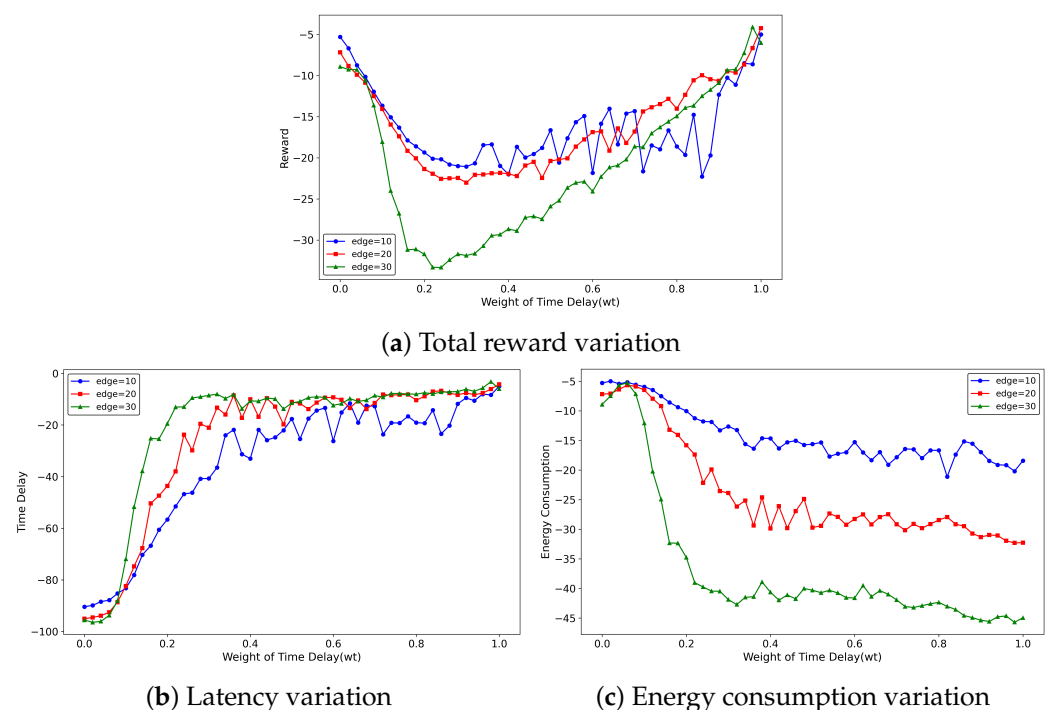


Figure 8. Impact of the number of edge servers on scheduling algorithm performance.

(2) Impact of the number of terminals on scheduling algorithm performance. To investigate the influence of the number of terminals on the performance of the scheduling algorithm, we fixed the number of edge servers and set the number of terminals to $\text{terminals} = [5, 10, 15]$, testing the algorithm's performance under different configurations.

From the results in Figure 9, it can be observed that the differences in latency, energy consumption, and total rewards across different numbers of terminals are not significant. This result demonstrates that our proposed MOOECN exhibits strong adaptability and robustness when handling computing task scheduling at varying scales. Whether for small-scale or large-scale computing tasks, MOOECN effectively balances latency and energy consumption, ensuring that the comprehensive reward of task scheduling remains at a high level. This adaptability primarily stems from the design of MOOECN's optimization algorithm, which dynamically adjusts scheduling strategies based on task requirements and resource distribution, thereby achieving efficient resource allocation and task execution across different scenarios. Consequently, MOOECN has broad applicability in practical applications and can meet diverse computing resource scheduling needs.

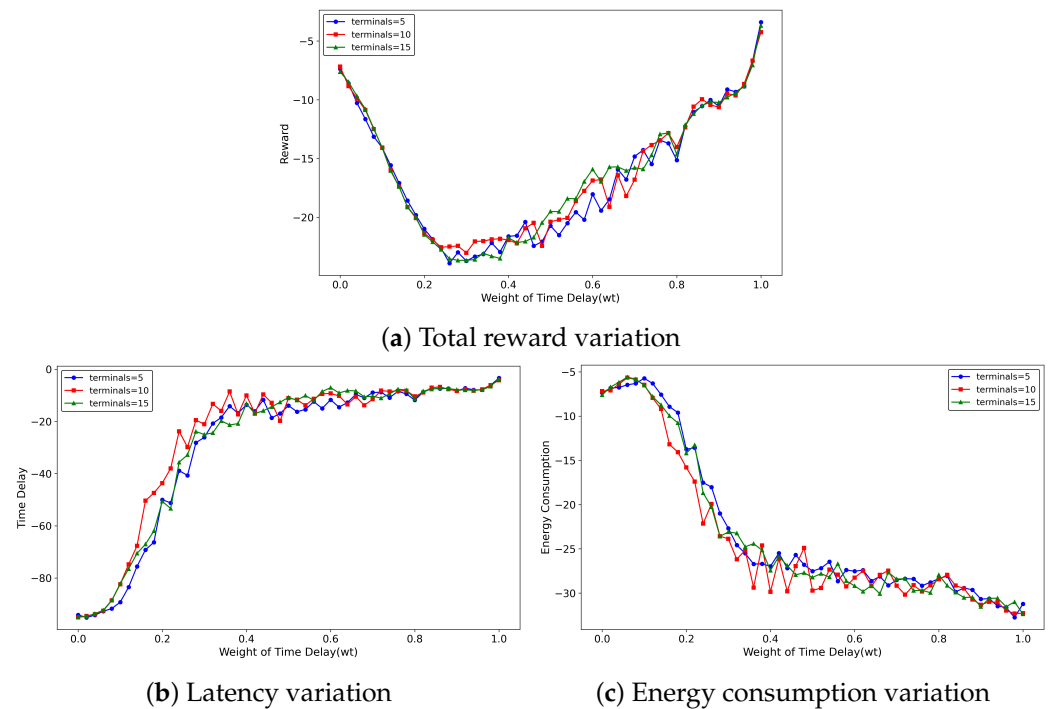


Figure 9. Impact of the number of terminals on scheduling algorithm performance.

(3) The impact of task size range on algorithm performance. To investigate the impact of task size range on algorithm performance, we fix the number of servers and terminals, as well as the minimum task size at 0.1 MB. We set the maximum task size to [50 MB, 100 MB, 150 MB] and test the performance of the algorithm under these settings.

From the results shown in Figure 10, it can be observed that smaller maximum task sizes generally lead to higher reward values in terms of overall performance, delay, and energy consumption. This is because small tasks are easier to schedule across edge servers and end devices. However, large tasks are also common in real-world scenarios. Table 6 presents the Pareto hypervolume under different maximum task size settings. The results indicate that smaller task sizes tend to yield better Pareto hypervolume performance. Task sizes are generated using an exponential distribution, which reflects realistic workload patterns with mostly small tasks and occasional large ones. Upper and lower bounds are applied to ensure simulation stability and prevent extreme values that could cause numerical issues or exceed system capabilities. The range of task sizes has a significant impact on both resource scheduling strategies and key performance metrics such as latency and energy consumption. A larger upper bound allows for testing under high-load conditions but may result in increased delays and network pressure due to unprocessed tasks. On the other hand, reducing the overall task size may support lightweight scheduling studies but can increase scheduling frequency and overhead. In addition, the width of the task size interval affects the training of reinforcement learning policies. An overly wide range may hinder convergence, while a narrow range may limit the diversity of experimental outcomes.

Table 6. Comparison of Pareto volume under different maximum task sizes.

Task Maxsize	50 MB	100 MB	150 MB
Pareto Hypervolume	3609.25	3303.88	2252

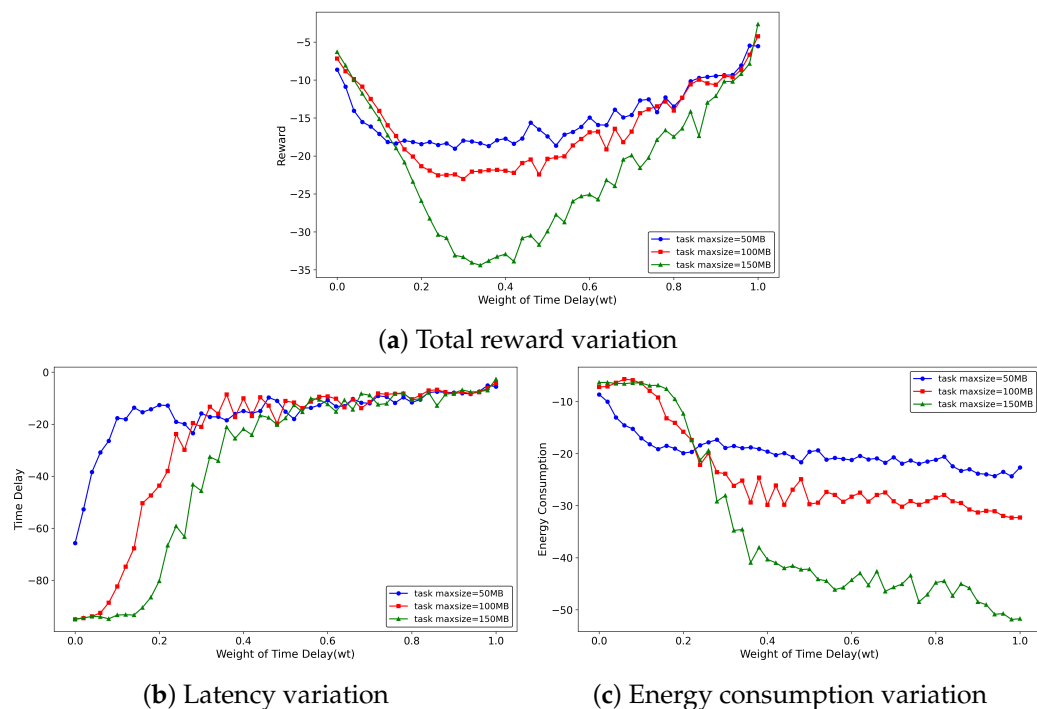


Figure 10. The impact of task size range on the performance of scheduling algorithms.

4.3. Evaluation on Real-Life Use Cases

To enhance the practical applicability and validity of our study, we construct a dedicated hardware experimental platform in this section to closely emulate real-world deployment scenarios.

4.3.1. Experimental Environment

To construct the edge computing environment, we set up cloud servers, edge servers, and end devices as computing nodes. These devices are all equipped with CPU chips but differ in architecture. Additionally, some devices are further equipped with GPU and NPU chips, resulting in heterogeneous computing resources.

For the workloads to be scheduled, each task is modeled as a directed acyclic graph (DAG) composed of microservices, more realistically reflecting the pipeline execution characteristics of modern edge applications. We designed three typical types of DAG workflows, corresponding to AI-intensive, memory-intensive, and disk-intensive resource demands. Specifically, we constructed 5 AI-intensive DAGs, 10 memory-intensive DAGs, and 10 disk-intensive DAGs, with each category packaged into independently deployable container images.

AI-intensive DAGs consist of multiple computation-heavy microservice stages. A typical workflow includes data preprocessing, lightweight deep learning model inference (e.g., quantized CNN or Transformer), and result postprocessing. Data tensors are passed between stages via memory or low-latency message channels, primarily consuming floating-point computational resources. These workflows impose high demands on the node's AI acceleration capabilities (e.g., GPU/NPU compute power) and memory bandwidth, while exhibiting low dependence on disk I/O.

Memory-intensive DAGs contain a series of microservices that require loading and processing large volumes of intermediate data at runtime (e.g., feature caching, real-time aggregation). Intermediate results are passed between microservices through shared memory, placing high demands on the node's CPU memory capacity and bandwidth. The DAG structure typically manifests as multi-stage serial or lightly parallel execution.

Disk-intensive DAGs are composed of microservices that frequently read from and write to local storage, such as chunked reading of large files, segment-by-segment processing, and result persistence. Microservices exchange data via the file system, leading to sustained high I/O load and imposing significant pressure on disk throughput and I/O latency.

These three categories of DAG workflows collectively simulate the typical characteristics of application structural complexity and diverse resource demands in realistic edge scenarios, providing an effective benchmark for evaluating the performance of scheduling algorithms in heterogeneous, dependency-aware environments.

4.3.2. Experimental Equipment Information

The experimental setup consists of three components: a cloud server, edge servers, and end devices. The specific hardware configurations are as follows:

- **Cloud Server:** One server equipped with two Intel Xeon Gold 5318Y processors (Intel Corporation, Santa Clara, CA, USA) with a base frequency of 2.1 GHz, six NVIDIA A6000 GPUs (NVIDIA Corporation, Santa Clara, CA, USA), and 8 TB of hard disk storage.
- **Edge Servers:** Five NVIDIA Jetson AGX Orin Developer Kits (NVIDIA Corporation, Santa Clara, CA, USA). Each unit features a 12-core ARM Cortex-A78AE CPU (Arm Limited, Cambridge, UK) running at 2.0 GHz, 32 GB of LPDDR5 memory, and an integrated Ampere GPU capable of delivering up to 200 TOPS (INT8). Storage is provided via 64 GB microSD cards.
- **End Devices:** Eight Raspberry Pi 5 boards (Raspberry Pi Ltd, Cambridge, UK). Each board is equipped with a quad-core ARM Cortex-A76 CPU (Arm Limited, Cambridge, UK) running at 2.4 GHz, 8 GB of RAM, and 64 GB of microSD card storage.

4.3.3. Evaluation Metrics

- **Average delay:** The arithmetic mean of the time interval from the moment a task is submitted by the cloud server to the system until its final execution result is successfully returned to the cloud server, calculated over all successfully completed tasks. This metric reflects the system's overall responsiveness in processing mixed workloads across heterogeneous edge resources.
- **Average energy consumption:** The average electrical energy consumed per task during its execution is computed as the total energy consumed by all participating edge servers and end devices throughout the entire batch execution period, divided by the number of successfully completed tasks. Specifically, the energy consumption of the five NVIDIA Jetson AGX Orin edge servers is measured using their built-in power sensors to collect board-level power, which is then integrated over the task execution duration. For the eight Raspberry Pi 5 end devices, a calibrated USB power meter records voltage and current during operation; instantaneous power is calculated and integrated to obtain the energy consumption. This metric measures the system's energy efficiency in completing individual tasks.
- **Task completion rate:** This is the percentage of submitted tasks that are successfully completed and return results. A task is considered to have failed if it cannot return a valid result due to reasons such as resource insufficiency or node failure. This metric reflects the robustness and reliability of the scheduling policy in a real-world heterogeneous environment.
- **Resource utilization:** This is the comprehensive average utilization rate of CPU and memory resources across all participating nodes (including edge servers and end devices) during task execution. For Jetson nodes, CPU utilization and memory usage

are obtained via tegrastats. For Raspberry Pi nodes, the corresponding metrics are collected using the psutil library. The final value is a spatio-temporal average, taken over all nodes and all sampling instants. This metric characterizes the system's efficiency in utilizing heterogeneous computing resources, with a higher value indicating less resource waste.

4.3.4. Experimental Results

As shown in Table 7, we conducted experiments under the above conditions, and the results are shown in the table below. The “ $\pm$ ” following each value in the table represents the mean $\pm$ standard deviation, calculated from 20 independent experimental runs, reflecting both the typical performance and operational stability of the algorithms. The overall trend clearly shows that the proposed MOOECN algorithm significantly outperforms all baseline methods across all four core metrics. It achieves an average delay of 18.7 s, which is 7.9% shorter than the second-best heuristic method and 51.6% shorter than the random strategy. Its average energy consumption is as low as 135.6 J, demonstrating substantial energy savings. Furthermore, MOOECN achieves a task completion rate of 93.2% and a resource utilization rate of 72.4%. Notably, it exhibits the smallest standard deviations across all metrics, indicating that its scheduling policy is not only superior in performance but also highly stable and robust.

Table 7. Performance comparison of different scheduling algorithms.

Algorithm	Average Delay (s)	Average Energy Consumption (J)	Task Completion Rate (%)	Resource Utilization (%)
Random	38.6 $\pm$ 12.4	215.3 $\pm$ 48.7	68.2 $\pm$ 9.5	41.3 $\pm$ 14.2
Greedy	29.4 $\pm$ 9.8	182.6 $\pm$ 36.5	76.5 $\pm$ 7.8	52.7 $\pm$ 11.3
MAB	25.8 $\pm$ 8.2	168.4 $\pm$ 31.2	81.3 $\pm$ 6.4	58.9 $\pm$ 9.7
DQN	23.5 $\pm$ 7.1	156.8 $\pm$ 28.4	84.7 $\pm$ 5.6	63.2 $\pm$ 8.5
SAC	21.9 $\pm$ 6.5	149.2 $\pm$ 25.8	87.6 $\pm$ 4.3	66.8 $\pm$ 7.9
Heuristic	20.3 $\pm$ 5.9	142.5 $\pm$ 23.1	89.4 $\pm$ 3.7	69.1 $\pm$ 7.2
MOOECN	18.7 $\pm$ 4.8	135.6 $\pm$ 20.3	93.2 $\pm$ 2.5	72.4 $\pm$ 6.1

Bold values indicate the best results.

This advantage stems from MOOECN's joint modeling of the heterogeneity in edge environments and the dependency structure of DAG tasks: it can accurately identify AI-intensive tasks and schedule them to Jetson edge servers equipped with NPU acceleration, avoiding severe delays or failures on Raspberry Pi devices due to the lack of dedicated computing power; meanwhile, through a resource-aware feasibility-filtering mechanism, it effectively avoids real-world hardware bottlenecks such as out-of-memory errors and microSD I/O timeouts. In contrast, methods like random and greedy, lacking global environmental awareness, often misassign heavy workloads to resource-constrained end devices, resulting in high latency, high energy consumption, and low task completion rates. The experimental results fully validate the effectiveness of MOOECN in achieving multi-objective co-optimization of latency, energy efficiency, reliability, and resource utilization within a real cloud-controlled edge architecture.

4.4. Practical Deployment Challenges and Scalability Analysis

Although the proposed MOOECN scheduling algorithm has been validated in both simulations and real-world heterogeneous hardware testbeds for its effectiveness in optimizing task latency and energy consumption, several key challenges remain when deploying it in large-scale, highly dynamic edge computing networks.

First, regarding computational overhead and scalability, the current algorithm employs a centralized PPO-based policy for scheduling decisions. In our experimental testbed, the per-decision latency is approximately 5–8 ms, which is significantly shorter than the typical execution time of DAG tasks, making it practical for small- to medium-scale edge clusters. However, in ultra-large-scale deployments (e.g., hundreds of edge nodes), the dimensionality of the state space grows linearly with the number of nodes, potentially leading to a significant increase in policy inference latency and even causing scheduling bottlenecks. Future work could explore hierarchical scheduling architectures or policy distillation techniques to improve scalability.

Second, concerning adaptability to dynamic environments, although our experiments were conducted under controlled conditions with a fixed number of edge servers, the proposed MOOECN framework is inherently designed to support dynamic edge environments. Specifically, the task arrival model follows a Poisson process, effectively simulating the random and burst-like workloads found in real-world scenarios. The action space is designed to be extendable—when a new edge node joins the network, its resource state can simply be incorporated into the state vector, and the corresponding scheduling option can be added to the action space, without requiring any modification to the network architecture or retraining of the policy. During training, we conducted ablation studies using multiple configurations with varying numbers of edge nodes, enabling the policy to adapt to system states of different scales and thereby achieving generalization capability to changes in node count.

Finally, the proposed MOOECN framework demonstrates low decision-making overhead and strong practicality in small- to medium-scale heterogeneous edge environments. Moreover, its extendable action space and training strategy naturally accommodate dynamic node availability and Poisson-distributed task arrivals, laying a solid foundation for deployment in large-scale, dynamic edge computing infrastructures.

5. Conclusions

This paper proposes a multi-objective optimization-based edge computing network resource scheduling algorithm, MOOECN. In the context of microservice-based edge computing networks, we introduce a hybrid dynamic–static computing power measurement method. This method reduces the dimensionality of multi-dimensional computing resources using the entropy weight method and combines dynamic computing power metrics to form a comprehensive measurement of computing resources. This approach enhances the matching between computing nodes and computing service demands. To address the resource scheduling problem in edge computing networks, we model the resource scheduling issue as an MOMDP. By leveraging multi-objective reinforcement learning and proximal policy optimization algorithms, we achieve multi-objective optimization of task transmission latency and energy consumption in dynamic environments. Extensive experiments demonstrate that our method outperforms state-of-the-art baseline models and delivers superior performance improvements. In future work, we plan to develop lightweight scheduling algorithms tailored to edge computing networks to further enhance the efficiency of computing resource scheduling in such environments.

Author Contributions: W.L. conceived the initial idea, designed the overall framework, and was responsible for writing and revising the manuscript. J.Z. and Y.F. implemented the algorithms, conducted the simulation experiments, and analyzed the data. H.W. participated in the discussion and optimization of the algorithm design. S.L. and X.Z. participated in the experimental design and results discussion. Y.J. and X.L. supervised the project, secured funding, and reviewed and revised key content of the manuscript. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported by the 2024 Sci-Tech Innovation Project of China Tower (Nanjing) Science and Technology Innovation Center (grant number: CX-JS-20241114-0-16704), the Jiangsu Key Development Planning Project (grant number: BE2023004-2), the National Key R&D Program of China (grant number: 2023YFB2904000, 2023YFB2904004), and the Natural Science Foundation of Jiangsu Province (Higher Education Institutions) (grant number: 20KJA520001).

Data Availability Statement: Dataset available on request from the authors.

Conflicts of Interest: The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. Authors Wenrui Liu, Yichao Fei and Hai Wang are employed by the company China Tower Corporation Limited, who provided funding and technical support for the work. The funder had no role in the design of the study; in the collection, analysis, or interpretation of data, in the writing of the manuscript, or in the decision to publish the results.

References

1. Liu, X.; Sun, C.; Zhou, M.; Wu, C.; Peng, B.; Li, P. Reinforcement Learning-Based Multislot Double-Threshold Spectrum Sensing with Bayesian Fusion for Industrial Big Spectrum Data. *IEEE Trans. Ind. Inform.* **2021**, *17*, 3391–3400. [CrossRef]
2. Na, Z.; Li, B.; Liu, X.; Wang, J.; Zhang, M.; Liu, Y.; Mao, B. UAV-Based Wide-Area Internet of Things: An Integrated Deployment Architecture. *IEEE Netw.* **2021**, *35*, 122–128. [CrossRef]
3. Zhou, W.; Xia, J.; Zhou, F.; Fan, L.; Lei, X.; Nallanathan, A.; Karagiannidis, G.K. Profit Maximization for Cache-Enabled Vehicular Mobile Edge Computing Networks. *IEEE Trans. Veh. Technol.* **2023**, *72*, 13793–13798. [CrossRef]
4. Rajput, K.R.; Kulkarni, C.D.; Cho, B.; Wang, W.; Kim, I.K. EdgeFaaS Bench: Benchmarking Edge Devices Using Serverless Computing. In Proceedings of the 2022 IEEE International Conference on Edge Computing and Communications (EDGE), Barcelona, Spain, 11–15 July 2022; pp. 93–103. [CrossRef]
5. Kiani, A.; Ansari, N. Edge Computing Aware NOMA for 5G Networks. *IEEE Internet Things J.* **2018**, *5*, 1299–1306. [CrossRef]
6. Du, Z.; Li, Z.; Duan, X.; Wang, J. Service Information Informing in Computing Aware Networking. In Proceedings of the 2022 International Conference on Service Science (ICSS), Shenzhen, China, 2–4 July 2022; pp. 125–130. [CrossRef]
7. Yao, H.; Duan, X.; Fu, Y. A computing-aware routing protocol for Computing Force Network. In Proceedings of the 2022 International Conference on Service Science (ICSS), Zhuhai, China, 13–15 May 2022; pp. 137–141. [CrossRef]
8. Wu, W.; Zhou, F.; Hu, R.Q.; Wang, B. Energy-Efficient Resource Allocation for Secure NOMA-Enabled Mobile Edge Computing Networks. *IEEE Trans. Commun.* **2020**, *68*, 493–505. [CrossRef]
9. Chen, L.; Fan, L.; Lei, X.; Duong, T.Q.; Nallanathan, A.; Karagiannidis, G.K. Relay-Assisted Federated Edge Learning: Performance Analysis and System Optimization. *IEEE Trans. Commun.* **2023**, *71*, 3387–3401. [CrossRef]
10. Zhao, R.; Zhu, F.; Tang, M.; He, L. Profit maximization in cache-aided intelligent computing networks. *Phys. Commun.* **2023**, *58*, 102065. [CrossRef]
11. Bertsekas, D. *Dynamic Programming and Optimal Control: Volume I*; Athena Scientific: Nashua, NH, USA, 2012; Volume 4.
12. Assila, B.; Kobbane, A.; El Koutbi, M. A Cournot Economic Pricing Model for Caching Resource Management in 5G Wireless Networks. In Proceedings of the 2018 14th International Wireless Communications & Mobile Computing Conference (IWCMC), Limassol, Cyprus, 25–29 June 2018; pp. 1345–1350. [CrossRef]
13. Ye, Y.; Shi, L.; Chu, X.; Hu, R.Q.; Lu, G. Resource Allocation in Backscatter-Assisted Wireless Powered MEC Networks with Limited MEC Computation Capacity. *IEEE Trans. Wirel. Commun.* **2022**, *21*, 10678–10694. [CrossRef]
14. Dinh, T.Q.; Tang, J.; La, Q.D.; Quek, T.Q.S. Offloading in Mobile Edge Computing: Task Allocation and Computational Frequency Scaling. *IEEE Trans. Commun.* **2017**, *65*, 3571–3584. [CrossRef]
15. Zheng, T.; Wan, J.; Zhang, J.; Jiang, C. Deep reinforcement learning-based workload scheduling for edge computing. *J. Cloud Comput.* **2022**, *11*, 3. [CrossRef]
16. Zhang, Y.; Zhou, Z.; Shi, Z.; Meng, L.; Zhang, Z. Online Scheduling Optimization for DAG-Based Requests Through Reinforcement Learning in Collaboration Edge Networks. *IEEE Access* **2020**, *8*, 72985–72996. [CrossRef]
17. Tuli, S.; Ilager, S.; Ramamohanarao, K.; Buyya, R. Dynamic Scheduling for Stochastic Edge-Cloud Computing Environments Using A3C Learning and Residual Recurrent Neural Networks. *IEEE Trans. Mob. Comput.* **2022**, *21*, 940–954. [CrossRef]
18. ETSI. *Mobile Edge Computing (MEC); Framework and Reference Architecture*; Technical Report ETSI GS MEC 003 V3.1.1; European Telecommunications Standards Institute (ETSI): Valbonne, France, 2022. Available online: https://www.etsi.org/deliver/etsi_gs/MEC/001_099/003/03.01.01_60/gs_MEC003v030101p.pdf (accessed on 5 October 2025).
19. Huawei. Computing Power Network (CPN) White Paper. Technical Report, 2023. Available online: <https://e.huawei.com/en/ict-insights> (accessed on 5 October 2025).

20. Amazon Web Services. AWS Panorama Developer Guide. 2023. Available online: <https://docs.aws.amazon.com/panorama/latest/dev/> (accessed on 5 October 2025).
21. Ren, J.; Lei, X.; Peng, Z.; Tang, X.; Dobre, O.A. RIS-Assisted Cooperative NOMA with SWIPT. *IEEE Wirel. Commun. Lett.* **2023**, *12*, 446–450. [\[CrossRef\]](#)
22. Lan, D.; Taherkordi, A.; Eliassen, F.; Liu, L.; Delbruel, S.; Dustdar, S.; Yang, Y. Task Partitioning and Orchestration on Heterogeneous Edge Platforms: The Case of Vision Applications. *IEEE Internet Things J.* **2022**, *9*, 7418–7432. [\[CrossRef\]](#)
23. Chen, Z.; Hu, J.; Chen, X.; Hu, J.; Zheng, X.; Min, G. Computation Offloading and Task Scheduling for DNN-Based Applications in Cloud-Edge Computing. *IEEE Access* **2020**, *8*, 115537–115547. [\[CrossRef\]](#)
24. Pu, L.; Chen, X.; Xu, J.; Fu, X. D2D Fogging: An Energy-Efficient and Incentive-Aware Task Offloading Framework via Network-assisted D2D Collaboration. *IEEE J. Sel. Areas Commun.* **2016**, *34*, 3887–3901. [\[CrossRef\]](#)
25. Chen, X.; Jiao, L.; Li, W.; Fu, X. Efficient Multi-User Computation Offloading for Mobile-Edge Cloud Computing. *IEEE/ACM Trans. Netw.* **2016**, *24*, 2795–2808. [\[CrossRef\]](#)
26. Wang, X.; Ning, Z.; Guo, S. Multi-Agent Imitation Learning for Pervasive Edge Computing: A Decentralized Computation Offloading Algorithm. *IEEE Trans. Parallel Distrib. Syst.* **2021**, *32*, 411–425. [\[CrossRef\]](#)
27. Chen, X.; Zhang, J.; Lin, B.; Chen, Z.; Wolter, K.; Min, G. Energy-Efficient Offloading for DNN-Based Smart IoT Systems in Cloud-Edge Environments. *IEEE Trans. Parallel Distrib. Syst.* **2022**, *33*, 683–697. [\[CrossRef\]](#)
28. Gao, H.; Wang, X.; Wei, W.; Al-Dulaimi, A.; Xu, Y. Com-DDPG: Task Offloading Based on Multiagent Reinforcement Learning for Information-Communication-Enhanced Mobile Edge Computing in the Internet of Vehicles. *IEEE Trans. Veh. Technol.* **2024**, *73*, 348–361. [\[CrossRef\]](#)
29. Peng, Q.; Wu, C.; Xia, Y.; Ma, Y.; Wang, X.; Jiang, N. DoSRA: A Decentralized Approach to Online Edge Task Scheduling and Resource Allocation. *IEEE Internet Things J.* **2022**, *9*, 4677–4692. [\[CrossRef\]](#)
30. Phuc, L.H.; Phan, L.A.; Kim, T. Traffic-Aware Horizontal Pod Autoscaler in Kubernetes-Based Edge Computing Infrastructure. *IEEE Access* **2022**, *10*, 18966–18977. [\[CrossRef\]](#)
31. Liu, T.; Ni, S.; Li, X.; Zhu, Y.; Kong, L.; Yang, Y. Deep Reinforcement Learning Based Approach for Online Service Placement and Computation Resource Allocation in Edge Computing. *IEEE Trans. Mob. Comput.* **2023**, *22*, 3870–3881. [\[CrossRef\]](#)
32. Zhou, H.; Jiang, K.; Liu, X.; Li, X.; Leung, V.C.M. Deep Reinforcement Learning for Energy-Efficient Computation Offloading in Mobile-Edge Computing. *IEEE Internet Things J.* **2022**, *9*, 1517–1530. [\[CrossRef\]](#)
33. Yang, Y.; Wang, S. EdgeOPT: A competitive algorithm for online parallel task scheduling with latency guarantee in mobile edge computing. *IEEE Trans. Commun.* **2024**, *72*, 7077–7092. [\[CrossRef\]](#)
34. Khoshvaght, P.; Haider, A.; Rahmani, A.M.; Rajabi, S.; Gharehchopogh, F.S.; Lansky, J.; Hosseinzadeh, M. A Self-Supervised Deep Reinforcement Learning for Zero-Shot Task Scheduling in Mobile Edge Computing Environments. *Ad Hoc Netw.* **2025**, *178*, 103977. [\[CrossRef\]](#)
35. Long, L.; Liu, Z.; Shen, J.; Jiang, Y. SecDS: A security-aware DAG task scheduling strategy for edge computing. *Future Gener. Comput. Syst.* **2025**, *166*, 107627. [\[CrossRef\]](#)
36. Xie, R.; Feng, L.; Tang, Q.; Zhu, H.; Huang, T.; Zhang, R.; Yu, F.R.; Xiong, Z. Priority-aware task scheduling in computing power network-enabled edge computing systems. *IEEE Trans. Netw. Sci. Eng.* **2025**, *12*, 3191–3205. [\[CrossRef\]](#)
37. Mokhtari, A.; Hossen, M.A.; Jamshidi, P.; Salehi, M.A. FELARE: Fair Scheduling of Machine Learning Tasks on Heterogeneous Edge Systems. In Proceedings of the 2022 IEEE 15th International Conference on Cloud Computing (CLOUD), Barcelona, Spain, 10–16 July 2022; pp. 459–468. [\[CrossRef\]](#)
38. Zhang, M.; Cao, J.; Yang, L.; Zhang, L.; Sahni, Y.; Jiang, S. ENTS: An Edge-native Task Scheduling System for Collaborative Edge Computing. In Proceedings of the 2022 IEEE/ACM 7th Symposium on Edge Computing (SEC), Seattle, WA, USA, 5–8 December 2022; pp. 149–161. [\[CrossRef\]](#)
39. Ma, T.; Wang, M.; Zhao, W. Task scheduling considering multiple constraints in mobile edge computing. In Proceedings of the 2021 International Conference on Intelligent Computing, Automation and Systems (ICICAS), Chongqing, China, 29–31 December 2021; pp. 43–47. [\[CrossRef\]](#)
40. Gong, Q.; Xia, Y.; Zou, J.; Hou, Z.; Liu, Y. Enhancing Dynamic Constrained Multi-Objective Optimization with Multi-Centers Based Prediction. *IEEE Trans. Evol. Comput.* **2025**. [\[CrossRef\]](#)
41. Zheng, X.L.; Wang, L. A Collaborative Multiobjective Fruit Fly Optimization Algorithm for the Resource Constrained Unrelated Parallel Machine Green Scheduling Problem. *IEEE Trans. Syst. Man Cybern. Syst.* **2018**, *48*, 790–800. [\[CrossRef\]](#)
42. Wang, F.; Sun, J.; Gan, X.; Gong, D.; Wang, G.; Guo, Y. A Dynamic Interval Multi-Objective Evolutionary Algorithm Based on Multi-Task Learning and Inverse Mapping. *IEEE Trans. Evol. Comput.* **2025**. [\[CrossRef\]](#)
43. Li, J.; Shang, Y.; Qin, M.; Yang, Q.; Cheng, N.; Gao, W.; Kwak, K.S. Multiobjective Oriented Task Scheduling in Heterogeneous Mobile Edge Computing Networks. *IEEE Trans. Veh. Technol.* **2022**, *71*, 8955–8966. [\[CrossRef\]](#)
44. Pan, L.; Liu, X.; Jia, Z.; Xu, J.; Li, X. A Multi-Objective Clustering Evolutionary Algorithm for Multi-Workflow Computation Offloading in Mobile Edge Computing. *IEEE Trans. Cloud Comput.* **2023**, *11*, 1334–1351. [\[CrossRef\]](#)

45. Li, L.; Qiu, Q.; Xiao, Z.; Lin, Q.; Gu, J.; Ming, Z. A Two-Stage Hybrid Multi-Objective Optimization Evolutionary Algorithm for Computing Offloading in Sustainable Edge Computing. *IEEE Trans. Consum. Electron.* **2024**, *70*, 735–746. [[CrossRef](#)]
46. Al-Bakhrani, A.A.; Li, M.; Obaidat, M.S.; Amran, G.A. MOALF-UAV-MEC: Adaptive Multiobjective Optimization for UAV-Assisted Mobile Edge Computing in Dynamic IoT Environments. *IEEE Internet Things J.* **2025**, *12*, 20736–20756. [[CrossRef](#)]
47. Qiu, Q.; Ye, Y.; Li, L.; Xiao, Z.; Lin, Q.; Ming, Z. Joint computation offloading and service caching in Vehicular Edge Computing via a dynamic coevolutionary multiobjective optimization algorithm. *Expert Syst. Appl.* **2025**, *284*, 127821. [[CrossRef](#)]
48. Sunny, A. Joint Scheduling and Sensing Allocation in Energy Harvesting Sensor Networks with Fusion Centers. *IEEE J. Sel. Areas Commun.* **2016**, *34*, 3577–3589. [[CrossRef](#)]
49. Alipour-Fanid, A.; Dabaghchian, M.; Arora, R.; Zeng, K. Multiuser Scheduling in Centralized Cognitive Radio Networks: A Multi-Armed Bandit Approach. *IEEE Trans. Cogn. Commun. Netw.* **2022**, *8*, 1074–1091. [[CrossRef](#)]
50. Nandhakumar, A.R.; Baranwal, A.; Choudhary, P.; Golec, M.; Gill, S.S. EdgeAISim: A toolkit for simulation and modelling of AI models in edge computing environments. *Meas. Sens.* **2024**, *31*, 100939. [[CrossRef](#)]
51. Wang, Z.Y.; Pan, Q.K.; Gao, L.; Jing, X.L.; Sun, Q. A cooperative iterated greedy algorithm for the distributed flowshop group robust scheduling problem with uncertain processing times. *Swarm Evol. Comput.* **2023**, *79*, 101320. [[CrossRef](#)]
52. Burra, R.; Singh, C.; Kuri, J. Service scheduling for random requests with fixed waiting costs. *Perform. Eval.* **2022**, *155*, 102297. [[CrossRef](#)]
53. Liu, T.; Tang, L.; Wang, W.; Chen, Q.; Zeng, X. Digital-Twin-Assisted Task Offloading Based on Edge Collaboration in the Digital Twin Edge Network. *IEEE Internet Things J.* **2022**, *9*, 1427–1444. [[CrossRef](#)]
54. Hu, M.; Zhou, M.; Zhang, Z.; Zhang, L.; Li, Y. A novel disjunctive-graph-based meta-heuristic approach for multi-objective resource-constrained project scheduling problem with multi-skilled staff. *Swarm Evol. Comput.* **2025**, *95*, 101939. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.